# CrossExam 质询室

> 全国高校 Vibe Coding 开发挑战赛 · AI 创新应用挑战赛（赛道 10039） 技术文档 —— 意图控制策略 / 架构设计 / 可复现性论证

**多 Agent 对抗式面试沙盘：把简历里那句你其实答不上来的话，在投出去之前找出来。**

| 项 | 内容 |
| --- | --- |
| 作品名称 | CrossExam（质询室） |
| 仓库 | 见 README 顶部（公开仓库，细粒度提交，共 120 余次） |
| 技术栈 | Node 24 + TypeScript（服务端零构建）· Hono + SSE · node:sqlite · Vite + React 19 + Tailwind v4 |
| 交付日期 | 2026-11-18（省赛作品提交截止） |
| 文档版本 | 1.0 · 2026-10-26 |
| 演示前提 | 不需要 API Key、不需要数据库、不需要 Docker，一条命令起完整产品 |

## 本文档怎么读

这份文档不是功能说明书，它只回答评委评分表上的三个问题：

| 评分维度 | 权重 | 对应章节 |
| --- | --- | --- |
| 过程可复现性与演进链路 | 25% | 第四章（可复现性论证）、第五章（实测证据） |
| 方法创新性与意图控制 | 20% | 第二章（意图控制策略） |
| 工程质量与架构设计 | 15% | 第三章（架构设计） |
| 解题有效性与实用价值 | 25% | 第一章（问题定义）与第五章（真实模型实测） |
| 用户体验与产品精细度 | 15% | 3.8 节（前端形态）与第五章（真浏览器实测） |

三条阅读路径，按可用时间自选：

1. **只有 5 分钟**：读第一章的 1.3 与 1.4，再看第二章的 2.2 那张四层表。
2. **想验收**：直接跳第五章，那里全是可复跑的命令与实测数字。
3. **想审设计**：第二章讲「怎么让模型听话」，第三章讲「为什么这么切模块」，第四章讲「凭什么说它可复现」。

文中所有数字都来自本机实测或仓库内的可复跑命令，来源标在数字旁边； 没有实测支撑的判断会明确写成判断，而不是伪装成结论。

# 第一章　问题定义：简历里那句话，你答不上来

## 1.1 一个具体的失败场景

求职者张明（本文档全程使用的示例人物）简历上写着两句话：

> 主导重构订单系统，首屏加载从 2.4s 降到 0.8s。 主导引入 Redis 缓存，QPS 从 800 提升到 5000。

他投出去，进了面试。面试官问：「2.4s 到 0.8s，怎么测的？取的均值还是 P75？样本量多少？」

张明答不上来。他确实做过这个优化，但那个数字是同事报给他的；他没测过，也不记得口径。 于是面试官得到两个结论：这个人对数字不负责；这份简历不可信。

**问题不在于他会不会写代码，而在于简历上的每一个数字，都是一个等待被追问的主张，而他没在被追问之前演练过。**

这个场景有三个关键性质，直接决定了产品形态：

| 性质 | 含义 | 对产品的约束 |
| --- | --- | --- |
| 主张是可枚举的 | 一份简历能抽出 5–12 条「可被追问的主张」 | 不需要做通用对话，做「逐条质询」就够 |
| 追问是有方向的 | 面试官不会随机问，他们拆口径、拆归属、拆因果 | 追问策略可以被写成可复用的意图契约 |
| 反馈必须可执行 | 「你答得不好」没有价值，有价值的是「这句话改成什么」 | 输出必须回落到简历原文的具体片段 |

## 1.2 现有方案为什么不够

市面上的简历工具分三类，各自缺一块：

| 类型 | 代表 | 它能做什么 | 它不能做什么 |
| --- | --- | --- | --- |
| 简历生成 / 美化 | 各类在线简历编辑器 | 排版、模板、导出 | 不检验内容是否站得住 |
| AI 润色 | 通用大模型对话 | 把句子写得更好看 | 会替你编，且编得比原文更自信 |
| 模拟面试 | 题库式 / 通用 AI 面试官 | 出题、打分、给建议 | 不锚定你的简历原文，问题与你写的东西无关 |

第三类是最近的形态，也是本项目的近邻。核心缺陷一句话：**它的提问不携带证据。** 「谈谈你最有挑战的项目」对任何人成立，因此对任何人都不成立——它指不出你简历里哪一句站不住。

## 1.3 CrossExam 的解法：把「面试」变成「逐条主张的证据审计」

CrossExam 的输入是**一份简历 + 一份目标岗位 JD**，输出是**每条主张的判定 + 回落到原文的修改建议。** 中间的过程是 3–5 位人设不同的 AI 面试官，对每条主张并发追问。

一场完整的质询有五个核心阶段，全部由程序编排、全部可回放；手里有岗位 JD 时前面会多三次读 JD 的调用， 收尾还会多一次「轮到你了」把材料反着用一遍（合计九个阶段、九种调用）：

```
简历 + JD
   │
   ├─ ⓪ 读岗位要求  jd-extract / jd-merge / jd-match
   │                          1–8 份 JD → 一张要求清单 → 与主张逐条对照（答得上 / 答得含糊 / 简历里没写）
   │                          逐份抽要求、归并、对照各一次调用；没上传 JD 就整段跳过
   │
   ├─ ① 抽取主张   extract    从简历里抽出可被追问的主张，每条必须携带逐字引文
   │                          锚不到原文的主张当场丢弃（不是降级，是丢弃）
   │
   ├─ ② 并发追问   probe      3 位面试官按各自的意图契约同时提问
   │                          每轮：预算闸门 → 锚定闸门 → 去重闸门
   │
   ├─ ③ 求职者作答 answer     求职者 Agent 只能引用简历原文作答，编造的引文被丢弃
   │                          连续答不上来 → 该主张自动关停（不再浪费追问）
   │
   ├─ ④ 判定       verdict    每条主张给出：站得住 / 表述过度 / 证据缺失 / 与原文矛盾 / 无法回答
   │                          两条代码级收敛：全没答上来又判站得住 → 证据缺失；要证据却拿不出 → 证据缺失
   │
   ├─ ⑤ 改写建议   suggest    把「答不上来」翻译成「这句话该怎么改」
   │                          四道闸门：改前原文必须定位得到、不许引入新数字、不许等于原文
   │
   └─ ⑥ 轮到你了   counter    给用户几句可以反问 HR 的话
                              每句必须锚回简历原话或 JD 原文，锚不回当场丢弃；回答由沙盘代答并注明「模拟」
```

五种判定不是设计出来的修辞，而是**证据审计的五个可能结论**：

| 判定 | 含义 | 触发条件（产品语义） |
|---|---|---|
| 站得住 | 主张有原文支撑，且追问下给出了可核验的口径 | 答得上 + 能援引原文 |
| 表述过度 | 主张本身没错，但说法超出了证据能支撑的范围 | 答得上但暴露了口径不一致 |
| 证据缺失 | 主张可能成立，但拿不出任何可定位到简历的证据 | 模型判定（责任在简历写法）；或模型判 solid/contradicts 却给不出引文 → 代码降级 |
| 与原文矛盾 | 追问下的回答与简历原文冲突 | 回答援引的原文与主张指向的事实相反 |
| 无法回答 | 候选人被追问后明确表示拿不出、不记得、没做过 | 模型判定（责任在人，风险最高的一类） |

## 1.4 与近邻产品的差异（不是形态差异，是判断依据的差异）

赛题禁止「用已有开源项目包装参赛」，所以这里做的是**能力对照**而非功能对照。立项时研究过一个同类产品 （本地优先的 AI 求职工作台，Apache-2.0，545 次提交），它把「防编造」做在**生成侧**：维护一份用户核实过的事实台账，只有核实过的才允许进入生成。

| 维度 | 事实台账式（生成侧防编造） | CrossExam（质询侧做压力测试） |
|---|---|---|
| 触发时机 | 写简历之前 | 简历写完之后、投出去之前 |
| 核心动作 | 拦截：不许写没核实过的事实 | 质询：写下的话经不经得起追问 |
| 输出物 | 一份更安全的简历 | 一份带判定与改写建议的审计报告 |
| 对模型的要求 | 不许发挥 | 必须发挥（扮演不同面试官施压） |
| 失败模式 | 台账太严，简历写不出来 | 追问太空，等于没问 |

两者互补而非重叠：台账保证「写下来的都是真的」，质询保证「写下来的都答得上」。选后者的理由是它多一个性质： **追问可以被证伪**——一次提问如果锚不到简历原文，它就是一条可被程序判定为无效的提问。这是第二章全部控制手段的起点。

### 1.5 谁会用、用在什么时候

| 用户 | 场景 | 得到的价值 |
| --- | --- | --- |
| 应届生 / 转行者 | 简历投出去石沉大海，不知道问题在哪 | 在投递前发现「这句我答不上来」 |
| 有经验但久未面试的人 | 项目是两年前做的，细节忘了 | 被追问后知道该补哪段记忆、该改哪句话 |
| 简历辅导 / 就业指导老师 | 批量看学生简历 | 一份可下载的审计报告，指出具体句子 |
| 面试教练 / 训练营 | 做模拟面试 | 用不同人设反复施压，且过程可复现 |

它不是「AI 帮你写简历」，而是「AI 帮你找出简历里最脆的那一句」。这带来一个直接约束：**报告必须定位到字。** 「描述不够具体」是无效反馈；「第 81–108 字这句『首屏加载从 2.4s 降到 0.8s』答不出测量口径」才是有效反馈。

### 1.6 本章要点

- 问题是「简历上的数字经不经得起追问」，不是「简历写得好不好看」。
- 因此产品形态是逐条主张的质询，不是通用对话。
- 因此每条提问必须携带简历原文锚点——这是可证伪性的来源。
- 因此输出必须回落到原文的具体片段——这是实用价值的来源。

---

# 第二章　意图控制策略

赛题把 20% 的权重给了「方法创新性与意图控制」。这一章要回答的问题很具体：

> 当你有 3 个 Agent 在同一个沙盘里各自发挥时，**你凭什么说它们在你的意图之内行事?**

### 2.1 「在提示词里写要求」不是意图控制

大多数多 Agent 产品的控制手段是：在 System Prompt 里写一段要求，然后相信模型会遵守。 例如「每条提问最多追问 3 次」「你的提问必须来自简历原文」。

这在工程上是不成立的，理由是**提示词是建议，不是约束**：

| 你写的 | 模型实际可能做的 | 后果 |
| --- | --- | --- |
| 「最多追问 3 次」 | 第 4 次继续问 | 死循环，烧钱，演示卡死 |
| 「提问要来自简历原文」 | 编一段听起来像简历的话 | 幻觉式提问，用户被问懵 |
| 「一次只问一个问题」 | 问三个连在一起 | 用户不知道答哪个 |
| 「答不上来就停止追问」 | 继续换角度施压 | 用户被反复羞辱同一件事 |

更麻烦的是：**这些违规不会报错。**模型会返回格式完全合法的 JSON， 里面装着一条越界的提问；如果你只校验 JSON 结构，它会一路通过，直到用户看见。

所以 CrossExam 的设计原则是：

> **凡是可以被程序判定的约束，就不要交给模型自觉。** 模型负责「想出好问题」，程序负责「保证这个问题合规」。

### 2.2 四层意图控制：从声明到收敛

本项目把意图控制拆成四层，每层有明确的职责边界与失败表现。

| 层 | 名字 | 做什么 | 在哪实现 | 被拦下时用户看到什么 |
| --- | --- | --- | --- | --- |
| L1 声明 | 意图契约 Intent Contract | 每个 Agent 的 System Prompt 是结构化 JSON，可解释、可版本化、可 diff | `packages/shared/src/contract.ts` | 界面上可展开查看契约全文与指纹 |
| L2 校验 | 结构 + 锚定 | 模型输出必须过 zod schema；每条提问的引文必须能在简历里逐字定位 | `packages/shared/src/schemas.ts`、anchor.ts | 被拦下的提问列在「被拦下的追问」里，附原因 |
| L3 记账 | 追问预算 Probe Budget | 追问次数由代码记账，不是由模型数数 | `packages/shared/src/budget.ts` | 主张卡片上出现「已关停」与关停理由 |

| 层 | 名字 | 做什么 | 在哪实现 | 被拦下时用户看到什么 |
| --- | --- | --- | --- | --- |
| L4 收敛 | 判定与建议的代码级收敛 | 模型给的判定与建议还要过一遍确定性规则 | `packages/server/src/orchestrator/verdict.ts`、suggest.ts | 判定卡片上标注「这条判定不是模型直接给的」 |

这四层的顺序很重要：**先声明，再校验，再记账，最后收敛。** 每一层都只依赖上一层已经确定的产物，因此整条链路是确定性的、可单测的、可在离线模式下复现的。

## 2.3 L1 声明层：意图契约是数据，不是文案

### 2.3.1 结构

每个面试官 Agent 的行为由一个结构化对象完全决定，而不是一段散文式提示词：

```
{
  "id": "tech",
  "version": 1,
  "persona": {
    "name": "技术面试官",
    "role": "tech",
    "seniority": "senior",
    "tone": "skeptical",
    "blurb": "资深工程师，冷静、只问细节"
  },
  "goal": "验证简历里的技术栈与项目描述是否真的出自本人之手，还是只写过 demo。",
  "tactics": [
    "追问技术选型的原因，而不是「用了什么」",
    "追问具体的失败与返工",
    "把简历里的一个数字拆开问口径"
  ],
  "constraints": [
    "每次提问必须携带能从简历里逐字找到的 quote",
    "引文至少 4 个字符",
    "一次只问一个问题",
    "只问简历里写过的东西，不要新造主张"
  ],
  "forbidden": [ ... ],
  "budget": {
    "maxRoundsPerClaim": 3,
    "maxProbesPerInterviewerPerClaim": 2,
    "maxTotalProbes": 60
  }
}
```

（以上为真实数据，来自 `PERSONA_PRESETS` 经 `buildContracts(['tech'], ...)` 生成， 技术面试官契约的指纹是 `042b385f925cf50f`，HR 是 `2375be34602e49f3`，业务主管是 `b2ed25a1d9d3995a`。）

### 2.3.2 为什么必须结构化

| 好处 | 具体表现 |
| --- | --- |
| 可解释 | 界面能把「它被允许做什么、被禁止做什么」摊开给人看，不用猜 |
| 可版本化 | `version` 字段 + 内容指纹，改一个字指纹就变 |
| 可 diff | 契约是 JSON，两次改动之间能看出改的是 tactics 还是 forbidden |
| 可对账 | 库里只存 `{id, version, digest}`，读侧按同样输入重算，指纹不一致就显示「契约已漂移」 |
| 可渲染 | System Prompt 由 `renderSystemPrompt(contract)` 确定性渲染，同样契约永远得到同一份提示词 |

第 4 条是本项目的一个明确取舍，也是设计上最值得讲的一处。

### 2.3.3 契约不落库全文，只落指纹

要做「契约可见」，最省事的做法是把契约 JSON 存进会话表，读出来显示。本项目没有这么做。

库里只存三样：

```
{ id: 'tech', version: 1, digest: '042b385f925cf50f' }
```

读侧每次按同样输入（面板 + 预算）**重新计算**契约全文，再用指纹与库里那一份对账：

```
const verified = match !== undefined
  && match.digest === digest
  && match.version === contract.version;
```

理由是「存副本」会把一句无法验证的断言混进产品：代码改了契约之后，老会话显示的仍是旧副本， 而旧副本已经不在任何地方运行了——**观众看到的和实际跑的不是同一份东西，且没有任何信号提示。**

重算 + 对账把这件事变成可判定的状态。界面上只有两种结论：

| 状态 | 界面文案 | 含义 |
| --- | --- | --- |
| 一致 | 与运行时一致 · 指纹 b2171c82 | 现在这份契约就是跑这一场时用的那份 |
| 漂移 | 契约已漂移 · 运行时 a1b2c3d4 → 现在 b2171c82 | 代码改过契约，这场会话用的是旧的 |
| 未开跑 | 尚未开跑 · 当前指纹 b2171c82 | 还没跑，无从对账 |

这个取舍的一般化结论写在 `findings.md` 的 F19 里： **凡是「实现里有一份、展示里又抄一份」的数据，都该改成「一份真值 + 一个指纹」。**

### 2.3.4 契约的渲染：System Prompt 的六节结构

模型实际拿到的不是上面那段 JSON，而是由它确定性渲染出的六节提示词。以技术面试官为例（真实输出，界面截图 10-system-prompt.png 就是它）：

| 节 | 作用 | 是否可被程序校验 |
| --- | --- | --- |
| 你这一场的目标 | 告诉它「为什么问」 | 否（属于意图） |
| 可用手法 | 给出三条具体战术，避免泛泛而问 | 否 |
| 硬约束（由程序强制校验，违反的提问会被丢弃） | 四条必须满足的条件 | **是**（L2 锚定 + 去重） |
| 禁止 | 四条禁区 | 部分 |
| 追问预算（由程序记账，你不需要自己记） | 三条上限 | **是**（L3 记账） |
| 输出要求 | 精确的 JSON 形状 | **是**（zod） |

注意「硬约束」与「追问预算」两节的标题措辞：它们明确告诉模型「这件事由程序负责」。 这不是修辞——它减少了模型在边界上的试探，因为模型知道越界会被直接丢弃，而不是被商量。

## 2.4 L2 校验层：证据锚定，让提问可证伪

### 2.4.1 规则

每条提问必须携带一个 `quote`，它是**从简历里逐字复制的片段**。程序做三件事：

1. 先做逐字查找（`source.indexOf(quote)`），命中即 `exact: true`；
2. 失败则做归一化查找（全半角标点映射 + 空白折叠 + 小写），命中则 `exact: false`；
3. 两者都失败 → 这条提问被丢弃，理由写进「被拦下的追问」，**不消耗预算。**

锚点的坐标由程序用 `source.slice(start, end)` 算出来，**不采信模型给的 start/end。** 这条决定记在决策表 D7：模型给的偏移量只当提示，坐标以引文回原文重新定位为准。

### 2.4.2 四种拒绝理由

| 拒绝码 | 中文标签 | 触发条件 |
| --- | --- | --- |
| empty-quote | 引文为空 | 模型没给 quote |
| too-short | 引文太短 | 少于 4 个字符（短引文能在简历里命中任何地方，等于没锚定） |
| quote-not-found | 引文在简历里找不到 | 模型改写了原文、补了标点、或编了一句 |
| out-of-range | 区间越界 | 计算出的区间超出简历长度 |

### 2.4.3 为什么这条规则是全部创新点的地基

「提问必须锚定原文」把一句主观判断（这个面试官问得好不好）变成了一句客观判断（**这个问题有没有证据**）。 它带来三个连锁效果：

- **幻觉式提问被物理消灭**：模型编一句「你在某大厂做过高并发」是过不了锚定的，因为简历里没有这句话。
- **界面可以拒显**：没有锚点的提问不会进入事件日志，因此也不会进入界面。
- **追问变得可评估**：每条提问都指向一个具体的简历片段，因此「这条主张被问了几次、问到什么程度」是可统计的。

第三个效果直接引出下一层。

## 2.5 L3 记账层：追问预算，让死循环在设计上不成立

### 2.5.1 问题

多 Agent 追问最常见的失败形态是**不收敛**：A 问完 B 换个角度问，B 问完 A 再追一层， 每一轮都「还有可以问的」，于是永远停不下来。提示词里写「最多 3 次」拦不住它。

### 2.5.2 做法

把追问次数做成一个**不可变的账本**（`packages/shared/src/budget.ts`），每次提问前必须过闸门，通过才记账：

```
export interface BudgetConfig {
  maxRoundsPerClaim: number;              // 同一条主张全场最多追问几轮
  maxProbesPerInterviewerPerClaim: number; // 每位面试官对同一条主张最多问几次
  maxTotalProbes: number;                  // 全场追问总次数上限
  maxConsecutiveUnanswerable: number;      // 连续几次答不上来就关停这条主张
}

export const DEFAULT_BUDGET: BudgetConfig = {
  maxRoundsPerClaim: 3,
  maxProbesPerInterviewerPerClaim: 2,
  maxTotalProbes: 60,
  maxConsecutiveUnanswerable: 2,
};
```

闸门 `canProbe(state, claimId, interviewerId)` 按固定顺序做五道检查，**任何一道不过就拒绝，并给出可直接展示给用户的中文理由**：

| 顺序 | 检查 | 拒绝理由（原文） |
|---|---|---|
| 1 | 主张是否存在 | 该主张不在本次沙盘中 |
| 2 | 主张是否已关停 | 关停理由，或「该主张已关停」 |
| 3 | 全场总次数 | 已达全场追问上限 60 次 |
| 4 | 该主张轮数 | 该主张已达追问轮数上限 3 轮 |
| 5 | 该面试官额度 | 该面试官对这条主张的提问额度已用完 |

顺序不是随意的：先判「这条主张还值不值得问」，再判「全场还有没有预算」，最后才判「这位面试官还有没有额度」。 这样拒绝理由总是最有信息量的那一条。

### 2.5.3 关停：连续答不上来就止损

账本还负责一件事：如果求职者连续 2 次答不上来同一条主张，这条主张被自动关停，理由写进账本：

> 已连续 2 次答不上来，继续追问没有信息增量

关停之后这条主张从「可追问清单」里消失（简历原文当然还是整份给模型），后续轮次不会再为它发起调用。 这条规则同时解决了两个问题：用户体验（不反复羞辱同一件事）与成本（不为已知结论烧 token）。

### 2.5.4 追问强度：换的是预算，不是人设

预算对用户可见，所以它必须可调：界面上有温和 / 常规 / 挑刺三档，落成三组不同的上限（每主张 2 / 3 / 4 轮、 每人每主张 1 / 2 / 2 次、全场 40 / 60 / 80 次）。三档只改**预算块**：人设、目标、手法、禁区一字不动。 理由是意图控制的一条边界——人设一改，两场之间的说法就不可比，「这个人比上次问得狠」这种判断失去意义； 而预算改了，契约指纹跟着变，界面会如实显示「当前指纹 cf9c6457」（挑刺档），换个档再换回来又回到原指纹。 档位随会话落库（`session.meta.difficulty`），重开一场历史会话仍看得出这一场问得多狠（决策 D46）。

代价很具体：契约是卡带指纹的输入，档位一变指纹就变，所以离线演示按三档各录了一份卡带（149 盘里占 3 份）。 这与「改提示词必须重录」是同一条规则，只是这次变的输入来自界面而不是代码。

### 2.5.5 编排层的收尾条件

编排循环的条件是：

```
while (rounds < maxRounds && probeableClaims(budget, panel).length > 0) { ... }
```

这里有一个容易写错的地方值得写进文档：一开始用的是 `openClaims(budget).length > 0`，而 `openClaims` 只表示「主张没有被 close」——额度用尽的主张仍然算 open，于是编排器会白烧一轮空的模型调用（live 模式是真金白银）。为此新增了 `probeableClaims(state, interviewerIds)`：**存在至少一位面试官能通过 `canProbe` 的主张。**

另外，如果一轮下来所有面试官都没有提出任何新问题（都被去重闸门拦了），编排器直接收尾并写一条如实记录的提示：

> 第 3 轮没有任何新的追问，质询到此结束

这条提示会出现在报告里，也会出现在界面的事件日志里——**收敛是可见的，不是悄悄结束的。**

## 2.6 L4 收敛层：模型给的判定与建议，还要再过一遍代码

### 2.6.1 判定阶段的两条收敛规则

模型判完之后，`enforceVerdict` 会用两条确定性规则复核（`verdict.ts`）：

| 规则 | 条件 | 处理 | 理由文案 |
|---|---|---|---|
| 全没答上来还判「站得住」 | 有追问，所有追问的回答都无效，且模型判「站得住」 | 降级为「证据缺失」 | 候选人对这条主张的追问没能落出口径，原文这一句不足以判为「站得住」，按「证据缺失」处理 |
| 要证据却拿不出 | 判为「站得住」或「与原文矛盾」，但证据引文一条都定位不到 | 降级为「证据缺失」 | 模型判为「站得住」但模型拿不出可定位到简历原文的证据，按「证据缺失」处理 |

第二条尤其重要：**「站得住」是一个需要证据的结论。**模型想判 solid，就必须拿出能在简历里定位到的引文；拿不出，这个判定就不成立。这与 L2 的锚定是同一个思想，只是从「提问」延伸到了「结论」。

第一条只做一件事：**防止过于乐观**，而不是替模型决定责任归属。代码手里只有一个布尔（这一场有没有答上来过），而模型看得见每一轮问答——它比这个布尔更清楚该算「简历写得太笼统」（证据缺失，改简历就行），还是「候选人自己拿不出东西」（无法回答，风险最高）。把「全体未答」一律改判「无法回答」的版本犯过一个真实错误：模型的理由里明明白白写着「只有结论、没有口径，属证据缺失」，界面上却挂着「无法回答」的徽章——理由是模型写的，结论是代码改的，两者当场矛盾（真实联网运行时才发现，见 5.6 第 5 条）。

为了让这两档可判别，`VERDICT_SYSTEM` 把判据写成了**责任在谁**：证据缺失 = 简历的错（这句话写得太笼统，改法是把口径写进去）；无法回答 = 人的错（本人承认不记得、举不出例子、没做过，面试现场会直接翻车）。拿不准时判「证据缺失」——它是一个可执行的结论，指向改简历；误判「无法回答」会让人以为这段经历该删掉。

被代码改过的判定会在界面上带一条琥珀色说明：「这条判定不是模型直接给的」+ 具体理由。截图 06-verdicts.png 里的 C5 就是这种情况。

### 2.6.2 建议阶段的四道闸门

改写建议是最容易被模型「发挥」的环节——它会顺手帮你加一个更漂亮的数字。因此 `suggest.ts` 设了四道闸门，任何一道不过，这条建议被拒绝并记录原因：

| 闸门 | 检查 | 拒绝理由 |
|---|---|---|
| 是否需要建议 | 判定为「站得住」的主张直接跳过（不调用模型、不消耗预算） | 判定为「站得住」，不需要改写 |
| 改前原文可定位 | `before` 必须能在简历里锚定到 | 建议里的「改前原文」在简历中定位不到，无法应用 |
| 不许引入新事实 | `after` 里的数字必须是简历里已有的数字 | 建议里出现了简历原文中不存在的数字，属于新增事实 |
| 必须真的改 | `after` 不能为空，也不能等于 `before` | 建议改写后与原文相同，等于没改 |

第二道闸门的具体实现是 `newNumbersIn(after, resumeText)`：用正则 `/[0-9]+(?:\.[0-9]+)?/g` 抠出建议里的所有数字，逐个检查是否在简历原文的数字集合里出现过；只要有一个新的，整条建议作废。

这条规则的效果可以在离线演示里直接看到：C2 的改写建议里出现的 `800` 与 `5000` 都来自简历原文，模型没有机会把「QPS 5000」改成「QPS 8000」。

### 2.6.3 归属由调用方决定

建议的归属（这条建议属于哪条主张）**一律由调用方传入**，模型自报的 `claimId` 不看。这一条在 live 实测里救过一次命：schema 曾把 `claimId` 列为必填，而提示词从没要求模型输出它，于是 9 条建议全部因「claimId: Required」被拒；离线演示恰好写了这个字段，把矛盾完整掩盖。修复后 `claimId` 改为 optional，并补了「模型不返回 claimId 也能出建议」的回归测试。

这个 bug 的教训写在 `findings.md` F23，也是本文档第 5.6 节要展开的「给模型输出造夹具的唯一真相是提示词里的输出要求」。

### 2.6.4 改写风格：调的是措辞收敛程度，不是事实边界

同一个毛病，有人想改得稳一点，有人想改得狠一点。所以改写阶段多了一个用户可选的档位：

| 档位 | 界面名 | 提示词里的取向 | 典型产出 |
|---|---|---|---|
| `conservative` | 保守 | 只做减法：删掉没依据的形容词，宁可句子变短变笨 | 删掉「大幅」「显著」这类词，保留原句式 |
| `normal` | 常规 | 该收敛的收敛，该补口径的补口径 | 给结果补上测量条件 |
| `bold` | 敢改 | 重写整句，但**不新增任何事实** | 把「优化了性能」重写成具体的「把首屏从 2.4s 降到 0.8s（同一套压测口径）」 |

三档**只改措辞的收敛程度**。2.6.2 那四道闸门在任何档位都不放宽，尤其第三条「不许引入新事实」—— `bold` 档最容易被模型当成「可以自己润色」的许可，所以它的档位说明里直接把「不新增任何事实」写死， 并且 `newNumbersIn` 照样在代码里逐条核对：模型说它没加数字不算数，代码认可才算数。

两个刻意的设计：

| 选择 | 为什么 |
|---|---|
| 档位**不进契约指纹** | 契约描述的是「谁在问」，风格描述的是「改得多狠」，两者是不同维度的意图。混进同一个指纹，会让「换了风格」看起来像「换了面试官」，历史场次之间就没法比了（追问强度的做法相反：它改的就是预算，就在契约里，见 2.5.4） |
| 档位随 `session.meta.style` 落库 | 报告是被带走的产物。抬头不写明这一场是「敢改」还是「保守」，两份报告摆在一起时，建议强弱不同就变得不可解释 |

代价与追问强度同源： `suggest` 提示词里带着档位，档位一换指纹就换，所以离线演示的 suggest 卡带要从 4 盘变成 12 盘（4 条建议 × 三档）。 这是「提示词是输入，输入变了素材就失效」这条规则第二次收账，也被记录在 ADR-0017 里。

## 2.7 过程可视化：流式分片是观感，事件日志才是真相

赛题的 UX 评分项明确点名了「流式、思考可视化」。本项目的处理方式是一条明确的分界：

| | 流式分片 | 阶段事件 |
|---|---|---|
| 内容 | 模型正在吐的字符 | 阶段开始 / 主张列表 / 追问列表 / 判定列表 |
| 落库 | **否** | 是（带 seq） |
| 可断线续传 | 否（刷新就没了） | 是（`?from=<seq>`） |
| 界面位置 | 卡片内的「正在作答」「正在定级」+ 过程面板的「实时输出」 | 阶段时间线 |

实现方式是给 LLM 客户端套一层 `withStreaming(inner, sink)`：各阶段的编排代码照旧 `await client.complete()`， 包装层把它改写成 `for await (inner.stream(req))`，把 `delta` 与 `reasoning` 分片送进 sink，最终结果取自 `done` 事件。 分片通过 SSE 的 `event: stream` 送出，**不带 id、不占 seq、不进事件表。**

理由写在 ADR-0004 里：分片每秒几十片，落库会让回放变慢、日志变吵，而分片拼出来的文本并不比 done 事件更可信。**能断线续传的只有阶段事件——这是「可复现」与「好看」之间的一条明确分界。**

顺带一个工程细节：分片携带的 `channel` 字段（例如 `answer:P1`、`verdict:C3`）**刻意不参与卡带指纹**， 因为指纹只对请求做哈希（system / messages / temperature / maxTokens / json）。 这条设计让「加流式」这件事不需要重录任何一盘离线卡带。

### 2.7.1 思考分片：接上会思考的模型，界面上多一行斜体

思考（`reasoning`）与正文（`delta`）是两个独立通道，用分片上的 `kind` 区分；界面渲染成两行—— 斜体那行是它正在「想」的，正下方那行是它要「说」的。要让这一行真的出现，需要三件事：

| 环节 | 做法 | 为什么 |
|---|---|---|
| 选谁思考 | 装配层 `withStageModels(inner, routes)`，按 `req.purpose` 换模型 | 判定才需要「停下来想」；追问一轮十几次调用，全交给思考模型会把一场从十几秒拉成几分钟（ADR-0005） |
| 换模型不发卡带 | `LlmRequest.model` 与 `channel` 一样不参与指纹 | 已录好的 149 盘演示卡带不必重录 |
| 没有 Key 也看得见 | 合成脚本把 `reasoning` 写进卡带（同一个通道） | 离线演示也要能证明「思考可视化」是产品能力，不是联网才有的彩蛋；卡带 `model` 固定 `scripted-demo`，一眼可辨 |

实测（2026-10-26，示例简历， `CROSSEXAM_REASONING_MODEL=deepseek-reasoner` 只路由 verdict）： 一场得到 **2,355 片思考分片 / 8 路 `verdict:*` 通道**，账本 `reasoningTokens` 大于 0，同一场正文分片 5,814 片，两条通道都不进事件表。 一条

要如实说明的代价：模型给的思考文本通常是**英文**（如 “The user asks: …”），这是模型侧行为，我们不翻译——硬翻译会让「思考」变成另一种编造。

### 2.8 本章要点

- 意图控制不是写提示词，是**四层可判定的机制**：声明（契约）→ 校验（锚定）→ 记账（预算）→ 收敛（规则）。
- 每条提问必须锚定简历原文，这是「提问可证伪」的地基，也是幻觉提问被物理消灭的原因。
- 追问预算由代码记账而非模型数数，因此死循环在**设计上**不成立，而不是「我们尽量避免了」。
- 判定与建议都要过代码复核：模型想判「站得住」必须拿出证据；想改简历不许引入新数字。
- 给用户的旋钮只调「程度」，不调「规则」：追问强度改预算不改人设，改写风格改措辞收敛不改事实边界。
- 契约不落库全文，只落指纹，读侧重算对账——一份真值 + 一个指纹，胜过两份迟早不同步的副本。

---

# 第三章　架构设计

### 3.1 一条约束决定了整个架构：服务端不设构建步骤

赛题 25% 是「过程可复现性与演进链路」。对一份要交给评委复现的项目来说，**每一个「先做 A 才能做 B」的步骤都是一个失败点**：

| 常见步骤 | 失败方式 |
| --- | --- |
| 先装编译器 / 编译原生依赖 | Windows 上 node-gyp 失败、Python 版本不对 |
| 先 build 后端再 run | 忘了 build、build 产物过期、build 报 TS 错误 |
| 先装 Docker | 评委机器没装或没权限 |
| 先申请 API Key | 没有卡、没有额度、区域不支持 |
| 先建数据库 | 忘了迁移、路径没权限 |

本项目把上表里能消灭的全部消灭了，得到一个极短的复现路径：

```
git clone  →  npm install  →  npm run preview  →  打开浏览器  →  点两下
```

其中最关键的一条决策是 **ADR-0001：服务端不设构建步骤，直接用 Node 原生 TypeScript 运行。**

#### 3.1.1 为什么这条能成立

Node 22.6+ 提供类型剥离（type stripping），可以直接运行 `.ts` 文件。但有一条众所周知的限制：**类型剥离默认不作用于 node_modules 内的文件。** npm workspaces 的包在 `node_modules/` 下是符号链接，因此理论上 `import '@crossexam/shared'` 会被当作「node_modules 内的模块」而拒绝剥离。

这个假设在 P1 阶段用一条最小验证钉死了（`.tmp/node-smoke.mjs`，纯 `node` 运行）：

| 验证项 | 结果 |
| --- | --- |
| `import '@crossexam/shared'` 拿到 46 个导出 | 成功 |
| 直接 import `replay.ts` / `cassette.ts` | 成功 |
| 跨包调用 `buildContracts(['tech','hr'], ...)` | 返回 2 条契约 |
| `chunkText` 分片后拼接 | 与原文逐字相等 |

原因是 Node 会先 realpath：符号链接解析后文件真实位置在 `node_modules` 之外，于是类型剥离正常生效。 **这条假设不是查文档查来的，是实测出来的**——这也是本文档反复强调的方法：凡是架构依赖的前提，都要有一条可复跑的最小验证。

#### 3.1.2 代价与边界

| 代价 | 处置 |
| --- | --- |
| 不能用 TS 的装饰器 / enum 等需要转译的语法 | 代码里本来也不用 |
| 类型检查与运行分离（运行时不做类型检查） | `npm run typecheck` 单独跑 `tsc --noEmit`，纳入提交门禁 |
| 生产部署不适合 | 本项目是本地优先的桌面级工具，不做服务端部署 |
| 前端仍需构建 | 前端是浏览器代码，必须 Vite 构建；但只有一条 `npm run build` |

## 3.2 模块划分

```
D:\CZB
├── packages/
│   ├── shared/        领域模型与纯函数内核（零依赖、无 IO、100% 可单测）
│   │   ├── domain.ts      领域类型：Claim / Probe / Answer / Verdict / Suggestion
│   │   ├── anchor.ts      证据锚定：引文 → 字符区间
│   │   ├── segment.ts     简历确定性切分：小节 / 条目 / 段落
│   │   ├── budget.ts      追问预算账本（不可变）
│   │   ├── contract.ts    意图契约与确定性提示词渲染
│   │   ├── schemas.ts     模型输出的 zod 契约与解析
│   │   └── util.ts        确定性 id、稳定序列化、哈希
│   │
│   ├── server/        编排与 IO
│   │   ├── llm/           LLM 适配层：live / record / replay 三态 + 卡带 + 流式包装
│   │   ├── orchestrator/  九阶段：extract / jd-extract / jd-merge / jd-match /
│   │   │                  probe / answer / verdict / suggest / counter + run 编排
│   │   ├── store/         node:sqlite 建表、迁移与仓储
│   │   ├── http/          Hono 路由 + SSE + 静态托管
│   │   ├── pipeline.ts    编排 → 落库 → 视图 的一体化入口
│   │   ├── report.ts      Markdown 报告渲染
│   │   └── bin/serve.ts   服务入口
│   │
│   └── web/           Vite + React 19 + Tailwind v4
│       ├── api.ts         HTTP 契约与 SSE 解析
│       ├── state.ts       事件流折叠成界面状态的纯函数
│       ├── labels.ts      文案（与 shared 的漂移由测试看守）
│       └── components/    五栏界面（右栏简历 / JD 分屏）
│
├── tools/
│   ├── dsh-export/    DSH 会话日志 → Prompt 链快照（提交物 #2 的生成器）
│   ├── smoke/         真进程冒烟（起服务、打接口、等 SSE 收尾）
│   ├── ui-shots/      无头浏览器截图器，兼作控制台报错门禁
│   ├── repro/         干净目录复现预演
│   └── docs/          Markdown → HTML → PDF 技术文档构建
│
├── fixtures/llm/      149 盘离线卡带（按提示词指纹寻址）
├── docs/              ADR / 技术文档 / 截图 / Prompt 链快照
└── data/              运行时 SQLite（已 gitignore，含用户简历隐私）
```

### 3.2.1 划分原则：按「依赖方向」而不是按「技术层次」

| 包 | 允许依赖 | 禁止依赖 | 理由 |
|---|---|---|---|
| shared | 只有 Node 标准库（`node:crypto`） | 任何 IO、任何网络、任何数据库 | 纯函数才好测；937 行源码配 489 行测试 |
| server | shared + Hono + zod | 不依赖 web | 服务端不知道前端长什么样 |
| web | React + 自己的 api.ts | **不 import @crossexam/shared** | 见 3.4 |

shared 里唯一的 IO 痕迹是 `util.ts` 的 `node:crypto`（算哈希），这一处恰好是 web 不能直接引 shared 的原因，也解释了为什么 web 要抄一份文案再用测试守住（3.4 节）。

## 3.3 LLM 适配层：一个接口，三种实现，一条卡带

### 3.3.1 接口

```
export type LlmMode = 'live' | 'record' | 'replay';

export interface LlmClient {
  readonly mode: LlmMode;
  complete(req: LlmRequest): Promise<LlmResult>;
  stream(req: LlmRequest): AsyncIterable<LlmStreamEvent>;
}
```

三个模式共用同一个请求结构，差别只在「结果从哪来」：

| 模式 | 结果来源 | 用途 |
| --- | --- | --- |
| live | 真实 HTTP 调用 | 联网跑，用户填了 Key 时 |
| record | 真实 HTTP 调用 + 落盘成卡带 | 录制演示素材 |
| replay | 从卡带读 | **默认模式**，评委零配置跑通全流程 |

### 3.3.2 卡带：用提示词指纹寻址

卡带（cassette）是一份 JSON 文件，文件名是提示词的指纹：

```
const digestFor = (req: LlmRequest) =>
  shortHash(stableStringify(shapeOf(req)), 24);

// shapeOf 只取这五项：
// { system, messages: [{role, content}], temperature, maxTokens, json }
```

**刻意不包含模型名与 purpose。**理由是换模型不该让整盘卡带失效——同一份提示词问出来的东西， 在演示意义上等价于同一段素材。这条决策也解释了为什么「加流式」不需要重录（channel 不在指纹里）。

回放客户端的失败方式是**响亮的**：

```
LlmError: 回放模式找不到这一条记录。用途「extract」的指纹是 b1206dc35699453a6dfbaa28，
请用 CROSSEXAM_MODE=record 联网跑一次把它录下来。
```

绝不静默降级到「跳过这一步」或「返回空结果」。这条选择在 `findings.md` F12 里有完整教训： 录制端第一版因为逐项容错，跑完 exit 0 但只写出 16/27 盘卡带，10 条回答全部缺失，stderr 干干净净。

### 3.3.3 记账包装：任何客户端都被套一层

```
withAccounting(inner, { db, sessionId, model })
```

它把每一次调用记进 `llm_call` 表：指纹、用途、模型、是否回放、四类 token、耗时。 界面顶部的「调用 29 次 · 回放 29 · token 23,533」就来自这里。 这层包装同时解决了两个问题：成本可见（live 模式知道花了多少）与**可复现性可见**（回放模式下「调用 N 次 / 回放 N 次」直接证明这一场没有联网）。

### 3.3.4 流式包装：只在需要直播时才套

```
const base = input.onStream === undefined
  ? input.client
  : withStreaming(input.client, input.onStream);
```

各阶段的编排代码完全不知道流式的存在（它们照旧 `await client.complete()`）。 好处是：JSON 路径（`?stream=0`）与 SSE 路径共用同一份编排逻辑，两条路径都被测试覆盖，且不会出现「流式版本和普通版本行为不一致」。

### 3.3.5 阶段路由：换模型是装配层的事

```
withStageModels(inner, { verdict: 'deepseek-reasoner' })
```

编排层只关心「问什么」，用哪个模型是装配层的事：`withStageModels` 按 `req.purpose` 在请求上带一个 `model` 字段， 各阶段的代码一行都不用改。默认只把**判定**交给思考模型（`CROSSEXAM_REASONING_STAGES` 可改）， 理由见 ADR-0005：判定才需要「停下来想」，而追问一轮要十几次调用。 `LlmRequest.model` 与 `channel` 一样**刻意不参与卡带指纹**，所以换模型不必重录任何一盘离线卡带。

## 3.4 前端形态：为什么 web 不 import shared

这是一个看起来违反直觉的决策（共享类型不是好事吗？），但有一条硬理由：

**shared 经 `util.ts` 依赖 `node:crypto`**，把它引进浏览器包会污染打包结果。 备选方案是把 shared 拆成一个零依赖的 core 包——这在架构上更干净，但会引入一次全仓库重构， 而本项目的时间预算要留给「意图控制」与「可复现性」。因此本项目选择：**类型与文案各抄一份，用测试守住漂移。**

### 3.4.1 漂移怎么守

| 会漂移的东西 | 守它的测试 | 断言方式 |
|---|---|---|
| 五种判定的中文标签 | `packages/web/test/labels.test.ts` | `toBe`（同一个对象，不是相同内容） |
| 七种主张类别 | 同上 | 逐项对照 + 顺序不漏不重 |
| 五类人设的名字与简介 | 同上 | 逐字对照 `PERSONA_PRESETS[role].persona` |
| 高亮区间的切分逻辑 | `packages/web/test/highlight.test.ts` | 9 条边界用例 |
| 事件折叠成界面状态 | `packages/web/test/state.test.ts` | 同一串事件喂给直播路径与历史路径，断言结果完全一致 |

最后一条是整个前端设计里最关键的一条：

> **直播与历史共用同一个纯函数折叠器**（ADR-0003）。

直播是把 SSE 事件一条条喂进去，历史是把库里的事件按 seq 读出来再喂进去， 两条路径的结果必须逐字段相同（除了 `phase` 由来源决定）。 这条测试保证了一个常被忽略的产品承诺：**刷新页面之后看到的，和刚才实时看到的，是同一场。**

### 3.4.2 界面结构：五栏 + 一条时间线

| 栏 | 内容 | 对应评分项 |
|---|---|---|
| 左：输入与过程 | 简历输入、岗位 JD（1–8 份）、节奏控制、运行模式、意图契约、阶段时间线、实时输出、统计 | 意图控制（契约可摊开） |
| 中上：主张与追问 | 每条主张一张卡片：原文引文、追问链、作答、判定、改写建议 | 实用价值（结论定位到字） |
| 中中：岗位要求对照 | 每条岗位要求一张卡片：要求正文、引文、「为什么这么判」、对应哪条主张 | 实用价值（岗位要求也落到字） |
| 中下：轮到你了 | 几句可以反问 HR 的话，每句带简历或 JD 原文依据，并写明回答是沙盘模拟的 | 实用价值（材料反着用一次） |
| 右：简历原文 / 岗位要求原文 | 左右分屏，两边都能点引文聚焦到具体字符区间，用的是同一套聚焦机制 | 证据锚定（两处锚都看得见） |
| 顶：状态条 | 模式、调用次数、回放次数、token、事件数、追问数 | 可复现性（回放可见） |

## 3.5 存储：node:sqlite，零原生依赖

选 `node:sqlite`（Node 24 内置）而不是 better-sqlite3，理由只有一个：**不引入需要本地编译的原生依赖。** 这条约束写进了 `AGENTS.md` 第 6 节：破坏「10 分钟复现」的依赖需要 ADR 才能进。

### 3.5.1 三个必须知道的差异（实测，记在 findings F8）

| 差异 | 后果 | 处置 |
|---|---|---|
| 外键约束按连接生效 | 漏发 PRAGMA 会安静留下孤儿数据 | `openDatabase` 里固定发 `PRAGMA foreign_keys = ON` |
| `prepare().all()` 返回 `unknown[]` | 每处用法都要断言 | 仓储层用一个私有辅助函数收口（断言会绕过类型检查，收口不会） |
| `PRAGMA user_version` 现成可用 | 不需要自建版本表 | 迁移直接用它做版本位 |

### 3.5.2 迁移：三条教训变成三条规则

| 规则 | 来源 |
|---|---|
| 连接即建表（readonly 除外） | F11：同进程测试全绿，真起服务第一次写入报 `no such table: session` |
| 迁移步骤冻结在自己时点，不引用当前 DDL | F24：v1→v2 若用当前 DDL，会一步建成 v3 表却把 user_version 写成 2 |
| 库版本比代码新时拒绝启动 | 防止老代码写新库，把数据写坏 |

当前 `SCHEMA_VERSION = 9`，表结构：

```
session(id, createdAt, updatedAt, llmMode, jobTitle, jdText, resumeText,
        resumeHash, status, budget_json, meta_json)
jd(session_id, id, ordinal, title, text)   PRIMARY KEY (session_id, id)
claim(session_id, id, ordinal, text, start, end, quote, exact, category,
      weight, status, verdict)          PRIMARY KEY (session_id, id)
probe(session_id, id, claim_id, interviewer_id, round, question, rationale,
      quote, start, end, parent_probe_id, requirement_id, jd_quote, jd_start,
      jd_end, jd_exact, answer_text, answer_mode, answer_cites_json, verdict)
                                        PRIMARY KEY (session_id, id)
requirement(session_id, id, ordinal, kind, text, keywords_json, jd_id,
      sources_json, quote, start, end, exact)  PRIMARY KEY (session_id, id)
requirement_match(session_id, requirement_id, claim_id, status, reason)
                                        PRIMARY KEY (session_id, requirement_id)
interviewer(id, version, digest, source, domain, contract_json,
      created_at, updated_at)           PRIMARY KEY (id)
revision(session_id, ordinal, claim_id, start, end, before, after, created_at)
                                        PRIMARY KEY (session_id, ordinal)
revised_resume(session_id, text, digest, accepted_json, pending_json,
      skipped_json, created_at, updated_at)    PRIMARY KEY (session_id)
supplement(session_id, claim_id, text, digest, created_at, updated_at)
                                        PRIMARY KEY (session_id, claim_id)
counter(session_id, id, ordinal, text, kind, reason, reply, source, jd_id,
      quote, start, end, exact, created_at)    PRIMARY KEY (session_id, id)
event(session_id, seq, kind, payload_json, createdAt)
llm_call(id, session_id, digest, purpose, model, replayed, usage_json, elapsedMs, createdAt)
```

会话相关的表都挂着 `session_id` 外键并级联删除：删掉一场会话不会留下孤儿行。`interviewer` 是唯一的**全局表**—— 它是一间面试官库，不属于任何一场会话；但一位面试官一旦被某场会话引用，它的 id、版本与指纹就冻结在那场会话的 `meta_json` 里，之后在库里改了或删了，旧会话的复现与对账仍然成立（第 3.6.6 节）。

复合主键（`session_id, id`）是 v2 迁移引入的。原因很具体：主张编号每场会话都从 `C1` 重新开始， 全局唯一主键会让同进程第二场会话直接撞 `UNIQUE constraint failed: claim.id`。

`revision` 的主键在 v8 从 `(session_id, claim_id)` 换成了 `(session_id, ordinal)`：P10 之后同一条主张可能同时落下两条改动—— 一条替换（改写建议）加一条零宽插入（用户补的那句真话），旧主键会让第二条把第一条覆盖掉。 `counter`（反问）与 `supplement`（补充说明）都是 v9/v8 新增的表，前者整批替换、后者只存那句话本身。

## 3.6 HTTP 层：一个端口，两种响应

| 接口 | 用途 |
|---|---|
| `GET /api/health` | 模式、跑法、模型、卡带目录，以及可选运行模式清单与「联网是否配好」 |
| `GET /api/models` | 联网模式下可用的模型清单（配不全时**一个网络包都不发**，直接返回缺哪个环境变量） |
| `GET /api/demo` | 内置示例简历与 JD |
| `POST /api/import` | 上传 PDF / Word / 文本，解析成简历正文（字节不存留；413 太大 / 415 格式不支持 / 422 解析不出文字） |
| `GET /api/contracts?panel=` | 意图契约全文 + 指纹（纯确定性，不调模型、不读库） |
| `GET /api/interviewers` | 面试官库：五位预设（只读）+ 自建的那些（带 id / version / digest / 渲染出的 System Prompt） |
| `POST /api/interviewers/draft` | 把一句话描述补成契约草稿（**不落库**）：`useModel:false` 走本地规则，模型失败自动退回本地并说明原因 |
| `POST /api/interviewers` | 保存自建面试官（预设不可改、不可删） |
| `DELETE /api/interviewers/:id` | 删掉一位自建面试官（预设返回 400，不存在返回 404） |
| `GET /api/templates` | 岗位模板库：12 个行业 × 3 个岗位，每条含 JD、示例简历、可追问点与推荐的两位面试官（只读，**不写任何表**） |

| 接口 | 用途 |
| --- | --- |
| `POST /api/templates/:id/apply` | 一键采用：把 JD 与示例简历填进投料台，并把模板自带的那位面试官装进库里（按模板 id 派生 id，**幂等**；返回 `created` 说明是新建还是复用） |
| `POST /api/sessions` | 建档（useDemo 或 resumeText，可指定 `mode`、`model`、`interviewers`，以及 `jds` ——1–8 份岗位 JD：这些选择都随会话落库） |
| `POST /api/sessions/:id/run` | 跑沙盘：默认 SSE 直播，`?stream=0` 走 JSON |
| `GET /api/sessions/:id/events?from=` | 事件续传 |
| `GET /api/sessions/:id/report.md` | 下载 Markdown 报告（采纳过改写就多一节「改前改后对照」） |
| `GET /api/sessions/:id/report.html?print=1` | 可打印 HTML 报告（`print=1` 时自动调 `window.print()`，PDF 由浏览器另存；采纳过才带并排对照表） |
| `GET /api/sessions/:id/report.doc` | Word 报告（同一份 HTML + `application/msword`，同上） |
| `POST /api/sessions/:id/supplements` | 记下某条主张的补充说明（一句真话）：只存那句话与指纹，**不存坐标**——插入点由主张的锚点现算 |
| `DELETE /api/sessions/:id/supplements/:claimId` | 删掉这条补充说明（没记过返回 404） |
| `GET /api/sessions/:id/revisions` | 当前采纳出来的那一版简历（没采纳过是 `null`） |
| `POST /api/sessions/:id/revisions` | 确认采纳：body 里的 `claims` 是**本次勾选的全部主张**，不是增量 |
| `DELETE /api/sessions/:id/revisions` | 撤销全部采纳，回到原文 |
| `GET /api/sessions/:id/resume.txt` | 改完的简历（纯文本，逐字就是要发出去的那一版） |
| `GET /api/sessions/:id/resume.html?print=1&marks=1` | 可打印简历（`marks=1` 把改动段标绿并附「本版改了什么」） |
| `GET /api/sessions/:id/resume.doc` | Word 简历（同一份 HTML + `application/msword`） |
| `GET /api/stats?session=` | 调用账本统计 |

### 3.6.1 SSE 的收尾承诺

SSE 有三个终止事件，任何一个发生都意味着这一场结束了：

| 事件 | 含义 |
| --- | --- |
| `result` | 正常跑完，附小结 |
| `fatal` | 致命失败（例如回放模式缺卡带），附原因 |
| 连接关闭 | 客户端断开 |

「一定会以成功或明确失败收尾」这条承诺由真进程冒烟守着（`tools/smoke/smoke.mjs` 会等 SSE 收尾并断言收到了 `result` 或 `fatal`）。

### 3.6.2 单端口托管前端

`mountWebApp(app, distDir)` 在 `/api/*` 之外挂静态托管 + SPA 回退。 这样评委只需要打开一个地址，不需要记两个端口，也不会有跨域问题。 服务入口用 `new URL('../../../web/dist', import.meta.url)` 定位前端产物，**因此从任何工作目录启动都能找到。**

### 3.6.3 报告导出：服务端出 HTML，PDF 交给浏览器

同一份 `SessionView` 有三种呈现：Markdown（`report.ts`，便于携带与 diff）、可打印 HTML（`report-html.ts`，便于阅读与打印）、 Word（`.doc` 后缀 + `application/msword`，便于接着改）。**服务端不生成 PDF**：那需要带一个无头浏览器或 PDF 库， 前者把「十分钟复现」变成赌评委机器上装了什么，后者违反零原生依赖。取舍详见 ADR-0006。

HTML 报告不引用任何外部资源，保存单文件即可离线打开；页脚写明「简历与报告都没有离开这台机器」。 打印脚本只在 `?print=1` 时注入——默认打开不会突然弹打印对话框（有测试守住）。

### 3.6.4 简历导入：解析成文本，但不留档

`POST /api/import` 只做一件事：把上传的字节变成简历正文，填进投料台的输入框，然后**字节就被丢掉**—— 不写临时文件、不入库、不进日志。于是「简历没有离开这台机器」在导入这条路径上也成立（上传的字节连磁盘都不落）。

解析按格式分三条路，每条都尽量少欠依赖：PDF 用 `unpdf`（纯 JS，零原生编译），`.docx` 自己解 zip （`node:zlib` 的 `inflateRawSync` 拆出 `word/document.xml`，不引 SDK），纯文本先按 UTF-8 解、出现替换字符再试 GBK （中文简历的现实

情况）。老版 `.doc` 是二进制复合文档，明确拒绝并提示另存为 `.docx`——**宁可给一句可执行的话，也不给一段看起来像正文的乱码。** 选型对比与三个反直觉的 PDF 行为见 ADR-0007、findings F39；导入器自己有 11 条单测，HTTP 分档另有 4 条。

### 3.6.5 运行模式与模型：逐场选，且随会话留档

三种模式（离线回放 / 联网直连 / 联网录制）在**页面里逐场选择**，选择结果写进会话行：`llmMode` 落模式，`meta.model` 落模型名。跑的时候按会话行建客户端——单一事实来源在库里，不需要内存里的映射表，翻回一周前的会话也能说清它当时是怎么跑出来的。

`GET /api/models` 只在显式选择联网后才被前端调用：先 `resolveLiveConfig`，缺 `LLM_BASE_URL` / `LLM_API_KEY` 就**一个网络包都不发**，直接把缺哪个变量告诉界面。拉不到清单不等于不能用——界面是 `input + datalist`，可以手填模型名；`/models` 返回什么也**不是可用性证据**（本机实测它只列出两款模型，而实际能调通四款）。见 D33、D34、F41。

这一块的提示文案曾经有五种状态（等清单 / 缺变量 / 报错 / 清单为空 / 没问题）挤在同一个 `<p>` 里，靠 `textContent` 判断就分不清「还在等」和「结论已经出来了」——点测脚本因此连续跑了 18 秒的循环，最后拿上一轮的文字做了断言（PASS 是假的）。处置是把状态收进 `packages/web/src/runmode.ts` 的两个纯函数：`modelNoteOf(list)` 返回该显示什么（没问题就返回 `null`），`isPendingNote(note)` 判断是不是还在等；界面与点测从此看同一个返回值，6 条单测守住。**「元素还在不在」本身就是一种状态，不能只靠文字内容去猜。**

### 3.6.6 自定义面试官：一句话补成契约，没有 Key 也能配

五位预设解决的是「计算机岗位」。可这个产品要能给任何行业用，就不能把面试官写死在代码里。 于是自建面试官的产物**不是一段自由文本，而是同一份** `IntentContract`——它和预设共用渲染、共用指纹、共用约束条款， 所以第 2 章讲的「意图可解释、可版本化、可 diff」对自建面试官一样成立。

链路是三段，每段都留了不联网的出口：

1. **描述 → 草稿**（`POST /api/interviewers/draft`，不落库）。模型返回的 JSON 必须过 `interviewerDraftSchema`（名称 1–24 字、目标 8–200 字、手法 2–5 条…），

过不了就不收。用户勾「只用本地规则」时走 `localDraftFromText`：纯规则、确定性、不联网，靠一张措辞表把话翻成手法（说了数字就追口径、说了「参与」就问边界、说了「好评」就问评选范围）。模型这条路失败时**自动退回本地并明说没走模型**，见 D36。

1. **草稿 → 库**（`POST /api/interviewers`）。落 `interviewer` 表（id + version + digest + 契约 JSON），是**全局库**而不是会话表。
2. **库 → 阵容 → 会话。** `POST /api/sessions` 的 `interviewers` 收 id 列表，解析出的 `panel` 写进 `meta.panel`。

**契约一旦被某场会话引用就冻结在会话里**：后来在库里改了、删了，旧会话的复现与对账仍然成立。

有个坑值得记下来：`resolvePanel` 只能把预算的**三个**字段交给契约。`maxConsecutiveUnanswerable` 是编排层的关停策略，不属于面试官契约；多给一个字段，同一场沙盘就会在「运行」与「对账」两处算出不同指纹，界面把没漂移的沙盘报成「契约漂移」（F42）。 预设不可改也不可删——它们的人设写在代码里，改预设等于悄悄改历史。上限 40 位，够用且防止界面被自己的库拖垮。 为什么不干脆让用户填一段 System Prompt 自由文本？因为那样就丢掉了可解释、可版本化、可对账、可约束这四件事。取舍详见 ADR-0008。

### 3.6.7 采纳改写：把建议变成一版真能发出去的简历

质询的产出如果只是「你这里讲不清楚」，用户还得自己去改。所以判定之后还有一步：**勾选想采纳的建议 → 确认 → 右栏换成改完的版本 → 导出 PDF / Word。**

这一步有三个容易做错的地方，选法是刻意的：

1. **接口只收 claimId，不收「把 A 换成 B」。**改前改后都从库里那条建议读——如果接口接受任意文本替换，它就等于一个注入面，任何人带着 sessionId 就能把任意内容写进简历。所以 `POST /api/sessions/:id/revisions` 的 body 只有一组主张 id。
2. **每次都从原文重算，采纳记录是集合不是叠加。**先采纳 C1 再采纳 C3，与一次采纳 [C1, C3]，结果**逐字节相同。**这样一来「撤销」不需要反向操作，`DELETE` 清空集合就从原文重算一遍；也就不存在「改了两遍、间距越改越乱」。
3. **改不动的要说出理由。**建议引文在原简历里锚不住（原文被用户改过）、这条主张本来就没有建议、用户点名了不存在的主张——三类都进 `skipped` 并给一句中文原因，界面上直接显示「勾了但没改成：…」，而不是静默少改一处。

导出复用报告那条路：服务端出 HTML，PDF 交给浏览器（ADR-0006），Word 是同一份 HTML + `.doc` 后缀。简历导出与报告导出**分两个渲染器**（`resume-export.ts` / `report-html.ts`）：报告是质询结论，简历是要发给 HR 的东西，混在一起会让简历带上满屏判定徽章。`marks=1` 时才把改动段标绿并附「本版改了什么（N 处）」——默认导出的就是干净的一版。改动坐标用的是 `movedSpans(applied)`：采纳会让后文整体平移，标记位置必须按累计位移重算，不能直接套原文坐标（见 `packages/shared/src/revise.ts`）。

### 3.6.8 岗位模板库：12 个行业 × 3 个岗位，点一下把料装好

空白投料台对不懂某个行业的人是死路：不知道该喂什么料，也不知道这个工具能问出什么。模板要解决的就是这十秒钟。

模板是**数据不是代码**：36 条 `JobTemplate`（`{id, industry, title, jd, resume, hints, presets, custom}`）放在 `packages/shared/src/templates/`，加一个行业只改数据文件，界面只读、规则不重复实现。三条选法是刻意的：**翻库不落库**（`GET /api/templates` 不写任何表——浏览一遍不该往全局面试官库里塞几十位）；**一键采用幂等**（模板自带的那位面试官，id 由 `customInterviewerId(name, template.id)` 派生，同一个模板永远同一个 id，点两次是复用，`created` 让界面说清是哪一种，F50）；**每条都必须留可追问的点**——示例简历刻意保留「下降明显」「参与抢救十余次」这类说法并带具体数字，否则一场质询全是「站得住」，恰好演示不出这个产品在干什么。

字数上下限与推荐阵容进常量 `TEMPLATE_LIMITS`，`templateProblems(list)` 是全库唯一校验入口：测试既拿它验 36 条全过，也拿它验「注入一处毛病会被点名」（F49）。取材合规上不做招聘平台爬虫——BOSS 直聘的 robots.txt 对 `User-agent: *` 写 `Disallow: /*?*`，而职位列表页全带查询参数；36 条模板的 JD 与示例简历全部原创。取舍见 ADR-0011、决策 D42。

### 3.6.9 岗位要求审阅：1–8 份 JD，两处锚点

上一节解决「不知道怎么喂料」，这一节解决「手里有料，但不知道够不够」。用户可以贴 1–8 份 JD（`POST /api/sessions` 的 `jds`，每份 40–6000 字），服务端把它读成两类东西。

**一是一张要求清单**（`requirement` 表）。三段链路：`jd-extract` 逐份 JD 并发抽要求（每份一次调用，上限 12 条），`jd-merge` 一次把所有 JD 的要求归并（同一件事写在两份 JD 里算一条，`sources` 记来源），`jd-match` 一次把全部要求和全部主张摆在一起逐条对照，给出「答得上 / 答得含糊 / 简历里没写」。归并和对照各只调一次是权衡后的选择：卡带按提示词指纹寻址，逐条调用会把 149 盘演示卡带再翻一倍，离线演示的成本与「提示词一改就全部失效」的风险都会失控。

**二是追问的第二处锚点。**对「答得上」和「答得含糊」的要求，追问可以带一个 `requirementId` **同时**带一段 JD 原文引文（`jdQuote`）；要求原文锚不回 JD 的，退回普通追问并在 rationale 里说明——宁可少一个锚，不编一个锚。三个刻意的选择：

- **双锚点不需要新机制。**`resolveAnchor(source, raw)` 从一开始就不关心 `source` 是简历还是 JD，

所以「一条追问同时挂简历片段与 JD 片段」只是再调一次同一个函数，双锚点因此与单锚点一样可验证、可回放。

- **「简历里没写」单独成栏，不生成追问。**面试官该追问的是简历里**说了但站不住**的话；

把「简历没写」也变成质问，产品就退化成一份岗位说明书审阅清单，而不是压力测试。

- **空 JD 是合法的。**没有 JD 时整条链路跳过，面试官照各自的契约追问（`jdProblems([])` 返回空数组）。

把「至少要有一份 JD」做成接口错误，纯简历场次就整条路走不通——「还没上传」该不该提醒是界面的事，不是数据校验的事（F55）。

一个只有落库才看得出的问题：双锚点是 `Probe` 的可选字段，第一版只把它写进事件流，业务表里没有对应列。于是重开一场历史会话，双锚点就消失了——报告与界面都查不出「这条追问是照着哪条要求问的」。修法是 schema v7 给 `probe` 补五列（`requirement_id` / `jd_quote` / `jd_start` / `jd_end` / `jd_exact`）并做一次表重建迁移。**「可选字段」不等于「可以不落库」：只要它在界面上显示过，它就是这份记录的一部分**（F54）。

界面按「左简历右 JD」分屏：右栏在简历原文与岗位要求原文之间切换；中栏多一段「岗位要求对照」，每条要求一张卡（要求正文、引文、「为什么这么判」、对应哪条主张），点引文右栏就聚焦到 JD 的那一段——和简历引文用的是同一套聚焦机制。取舍见 ADR-0012、决策 D43。

### 3.6.10 追问深化：把同一场质询读出四种结论

P10 的四项都在**同一份对话记录上换一个角度**：一律不新增判定口径，也一律从同一处算出来。

- **按类别看短板**（`packages/shared/src/shortfall.ts`）。把每条主张的判定按类别聚合，先排欠得最多的类别，

再列出最该补的 3 处。排序与结论文案只在 shared 里写一次，Markdown 报告、可打印 HTML 与界面读同一份——否则会出现「报告说最该补 C3、界面排第一的 C5」这种各算一遍的分裂（决策 D47）。

- **追问强度三档**（温和 / 常规 / 挑刺）。只换预算（每主张 2/3/4 轮、每人每主张 1/2/2 次、全场 40/60/80 次），

**不换人设**：人设一改，两场之间的档案链就断了。档位随会话落库（`session.meta.difficulty`），重开历史会话仍看得出这一场问得多狠。代价是契约指纹随档位变，所以卡带按三档各录了一份（ADR-0014、决策 D46）。

- **补充说明。**用户可以在任何一条主张上手写一句真话，当作这条主张的新证据。它在数据上就是一次**零宽插入**

（`start === end`、`before` 为空串），复用采纳改写的同一条通道——不必为「加一行」再发明一种写操作。它只落那句话本身、不落坐标，插入点由主张的锚点现算（坐标是原文的函数，存下来只会腐烂，见 ADR-0013）。

- **轮到你了。**跑完给出几句用户可以**反问 HR** 的话，每句都必须把依据锚回简历原话或 JD 原文；锚不回的当场丢弃，

在「被丢掉」抽屉里写明理由。回答由沙盘里的求职者 Agent 代答，界面上写明「沙盘模拟，不是真 HR 在答」——不然用户会把它当成真实面试答案背出去（ADR-0015）。

四条链路共用一个前提：**结论必须能由输入唯一确定**，所以它们都是纯函数，也都能在零网络下由仓库里的卡带复现。

### 3.6.11 改前改后对照：把改动摆回上下文里

采纳改写之后，界面与报告都要能回答「到底哪一句被改成了什么」。只把 before / after 一列一列摆出来是不够的：读者看不出这句改动落在简历的什么位置，也不知道前后文是什么。所以并排对照多摆一类行——

- **改动行**：左边是原文那一段，右边是改完的那一段；插入型改写的「原文」是空的，写「（新增一行）」。
- **上下文行**：改动之间那些**没动过**的原文也摆进来，两侧相同，超长时朝改动的方向截断（改动之前留尾巴、之后留开头）。

聚合只写一遍：`packages/shared/src/revise.ts` 的 `sideBySide(resumeText, applied)` 产出这些行， 报告（`report.ts` 里的三列表格）与界面（`RevisionBar` 的 `data-shot=revision-diff`）都读它—— 同一份结论在两处各算一遍，迟早会长出两个版本（F63）。这一节**只在真的采纳过之后出现**： 没采纳就不是「改前改后」，而是把同一份原文抄两遍（ADR-0016、决策 D50 / D51）； 章节编号也跟着走——没采纳时「复现方式」仍然是六。

界面里默认收起，点「并排看改动」才展开：它是核对时才看的东西，不该一直占着采纳栏的地方。

## 3.7 工程质量：门禁四道

`npm run verify` 是提交前必须全绿的一道门，它由四件事组成：

| 顺序 | 命令 | 证明什么 | 当前规模 |
|---|---|---|---|
| 1 | `npm run typecheck` | 类型对（`tsc --noEmit`，含各包） | 全仓库 |
| 2 | `npm run test` | 逻辑对（vitest） | 63 个文件 / 659 条断言 |
| 3 | `npm run smoke` | **产品能跑**（真起进程、真打 HTTP、真等 SSE） | 9 项 |
| 4 | `npm run build` | 前端能构建 | vite，42 modules（CSS 29.00 kB / JS 334.50 kB，gzip 103.16 kB） |

第 3 道是 `findings.md` F11 的直接产物：同进程的 `app.request()` 测试全绿，真起服务却第一次写入就报没有表。 **单元测试证明「函数逻辑对」，冒烟证明「产品能跑」——两句话之间隔着一个进程边界，而事故恰好长在边界上。**

除 verify 之外还有三个可选门禁，都做成了命令：

| 命令 | 作用 | 什么时候跑 |
|---|---|---|
| `npm run shots` | 无头浏览器点一遍全流程，出 24 张截图，**控制台有报错即失败** | 改了界面之后 |
| `npm run repro` | 干净目录 clone → install → verify → 起服跑一场，逐步计时 | 改复现路径之后 |
| `npm run seed:demo` | 重新生成 149 盘离线卡带并自检 | 改了提示词之后 |

### 3.7.1 代码规模与测试密度

| 模块 | 源码行 | 测试行 | 说明 |
|---|---|---|---|
| packages/shared/src | 3774 | 1633 | 纯函数与数据（含 36 条岗位模板、短板聚合、补充说明、反问依据、并排对照），测试密度最高 |
| packages/server/src | 9788 | 5397 | 编排 + IO + 导入解析 + 面试官库 + 岗位要求 + 反问 + 报告三种形态 + 采纳改写，含脚本化模型夹具 |
| packages/web/src | 6179 | 1733 | 折叠器、文案、工作室、模板选择器、要求对照、短板与反问栏、并排对照表（只计 .ts/.tsx，index.css 另 321 行） |
| tools/ | 3503 | 596 | 导出器 / 冒烟 / 截图 / 界面点测 / 复现 / 文档构建 / 零门槛启动器（只计 .mjs，测试目录另计） |
| **合计** | **23244** | **9359** | 测试/源码 ≈ 0.40 |

### 3.7.2 零门槛入口：双击就能用，判断逻辑可单测

「十分钟复现」是给会敲命令的人写的；第 7 条要求**完全没用过命令行的人也能装起来**。实现是三个入口脚本 （`启动.cmd` / `启动.sh` / `装个快捷方式.cmd`） + 纯函数模块 `tools/launcher/src/plan.mjs`，**不打包成 exe** （打包要本地编译，且我这台机器打包成功不代表别人机器上双击不被拦住，见 ADR-0010）。 `launch.mjs` 只碰文件与起进程，判断全在纯函数里—— 因为这类逻辑没法靠人肉点两遍验证： `launcherSteps({hasDependencies:false,hasWebBuild:false})` 必须是五步、全装好只剩三步（12 条测试，F46）。 两个只有真跑才露出来的细节：快捷方式**没给图标就绝不写** `IconLocation` （空串会让图标变白纸）； 启动前先探测 `/api/health` （命中就复用，第二个实例不撞端口，F47）。实测首次约 20 秒、之后约 2 秒。 仓库里 **0 个 TODO / FIXME / HACK、0 处** `as any` **、0 处** `@ts-ignore` （全仓库 grep 实测）。 console.log 只出现在两个 CLI 入口（serve.ts 与 seed-demo.ts）。

### 3.8 本章要点

- 服务端零构建是整条复现路径的地基，它成立的前提是被实测验证过的，不是查文档查来的。
- 模块按依赖方向切：shared 无 IO、server 不知前端、web 不引 shared（用测试守漂移）。
- LLM 适配层一个接口三种实现，卡带用提示词指纹寻址，缺卡带时响亮报错而不是静默降级。
- 直播与历史共用同一个折叠器，保证「刷新后看到的和刚才看到的是同一场」；门禁四道里真进程冒烟不可替代。

---

# 第四章　可复现性论证

赛题 25% 给「过程可复现性与演进链路」。这一章不讲愿景，只回答三个可核查的问题：

1. **别人能不能在我的机器之外跑起来？**（4.1–4.3）
2. **没有 API Key 能不能看到完整效果？**（4.4）
3. **我凭什么说这些提交是真做的，而不是最后一次性倒进去的？**（4.5–4.6）

### 4.1 复现路径：两条路，通到同一个界面

**路 A：不敲命令**（对应第 7 条要求）：装 Node LTS（https://nodejs.org/zh-cn/download）→ 下载 ZIP 解压到**无中文无空格**的目录 → 双击 `启动.cmd`（macOS / Linux 用 `./启动.sh`）。启动器自己判断该做什么（缺依赖就装、缺前端产物就构建、起服务后轮询 `/api/health` 通了再开浏览器、已有实例就直接复用），第一次约 20 秒、之后约 2 秒；细节与取舍见 3.7.2 与 ADR-0010。

**路 B：命令行**（开发者与评审的常规路径；`npm run launch` 与双击等价）

```
git clone <仓库地址>
cd crossexam
npm install
npm run preview        # 打开 http://127.0.0.1:8787 ，点「一键示例」，点「开始质询」
```

| 步骤 | 需要什么 | 不需要什么 |
|---|---|---|
| clone / 下载 ZIP | 网络 | 不需要账号（公开仓库） |
| install | Node ≥ 22.6（实测 24.14.0）、npm | 不需要 pnpm / yarn、不需要编译工具链 |
| preview | 磁盘可写 | 不需要 Docker、不需要 API Key、不需要数据库服务 |
| 浏览器操作 | 任意现代浏览器 | 不需要装任何客户端 |

#### 4.1.1 实测耗时（干净目录，`npm run repro` 的输出）

| 步骤 | 耗时 | 说明 |
|---|---|---|
| git clone | 24.6 s | 含 149 盘卡带与 24 张截图 |
| npm install | 38.8 s | 117 个包，来自 npmmirror |
| npm run verify | 48.4 s | 类型检查 + 659 条断言 + 冒烟 + 构建 |
| 起服 + 跑完一场 + 下载报告 | 1.2 s | 回放模式，46 ms 跑完整场 |
| **合计** | **120.8 s** | 距 10 分钟上限有 5 倍余量 |

这个表不是估的，是 `tools/repro/repro-check.mjs` 在一个**全新临时目录里 git clone 出来**的仓库上逐步计时得到的。同一台机器多次预演落在 34–121 秒之间：磁盘缓存冷热（`clone` 2.8 s 到 24.6 s 都出现过）与机器当时在不在忙都会影响，这张表记的是最近一次完整预演。慢的那一次（120.8 s）是机器同时在跑别的任务时测的，这也是诚实的区间上界。写文档时的第一条原则是：**复现耗时必须是实测数据，且必须由命令生成，不能手写**（决策 D16）。

### 4.2 为什么没有原生依赖

| 依赖 | 选择 | 如果换成常见的替代品会怎样 |
|---|---|---|
| 数据库 | `node:sqlite`（Node 内置） | better-sqlite3 需要 node-gyp + Python + VS Build Tools，Windows 上失败率极高 |
| 服务端框架 | Hono（纯 JS，零依赖链） | Express 也行，但 Hono 自带 `streamSSE` 且体积小 |

| 依赖 | 选择 | 如果换成常见的替代品会怎样 |
|---|---|---|
| 校验 | zod（纯 JS） | — |
| .env 载入 | `process.loadEnvFile`（Node ≥ 20.12 内置） | dotenv 会多一个包，且本项目只需要一行 |
| 浏览器驱动 | 系统 Edge/Chrome + CDP + Node 全局 WebSocket | Playwright 会下载 200 MB 浏览器 |
| 文档渲染 | 自写 md2html（零依赖） | pandoc 需要单独安装 |
| 截图 | CDP `Page.captureScreenshot` | — |

上表每一行都能对应到 `AGENTS.md` 第 6 节那条约束：「引入需要本地编译的原生依赖会毁掉 10 分钟复现，需要 ADR 才能破例」。 至今没有破例。

## 4.3 三个「静默失败」的坑，以及它们如何变成规则

可复现性的敌人往往不是「跑不起来」，而是「**看起来跑起来了，其实用的是别的东西**」。本项目踩过三个：

| 坑 | 现象 | 变成的规则 |
|---|---|---|
| 说明书写了 `cp .env.example .env`，程序从没读过它 | 填了 `CROSSEXAM_MODE=live` 仍是回放，配置静默失效 | 启动横幅必须打印「已载入哪个 .env」或「没有 .env」 |
| 端口上挂着旧进程 | live 实测报「0 分片」，其实测的是流式功能落地**之前**的旧服务 | 一次性实测起服务前必须自检端口为空，并核对写入的是自己的临时库 |
| 通用 .gitignore 模板的 `bin/` 段 | 源码被静默忽略，P0 提交漏掉了导出器 CLI | 忽略规则要**正反各测一遍**：密钥被挡住 + 源码没被挡住 |

第二、三条的完整复盘在 `findings.md` F22 与 F6/F10。它们共享同一个教训：

> **退出码为 0 不代表事情办成了。凡是「静默忽略」的机制，都要在事后做一次正面确认。**

## 4.4 离线回放：没有 Key 也能看到完整效果

### 4.4.1 卡带机制

回放模式下，每一次模型调用的「问题」被哈希成一个 24 位指纹，**答案**就存在以该指纹命名的 JSON 文件里：

```
fixtures/llm/
├── 0a3f...c2.json    extract 阶段的回答
├── 1b7e...d9.json    probe 阶段某位面试官的回答
└── ...               共 149 盘
```

指纹只对请求的五项内容做哈希（system / messages / temperature / maxTokens / json）， 因此**改了提示词，指纹就变，旧卡带自然失效并响亮报错。**这既是一个约束，也是一道门禁（4.4.3）。

### 4.4.2 149 盘卡带覆盖了什么

| 阶段 | 卡带数 | 说明 |
|---|---|---|
| extract | 4 | 抽主张：第一场 + 第二场，各按三档追问强度的提示词差异分开录 |
| jd-extract / jd-merge / jd-match | 各 1 / 1 / 4 | 读一遍内置示例 JD：抽要求 → 归并 → 与主张逐条对照 |
| probe | 78 | 3 位面试官 × 三档追问强度 × 两场（每场轮数不同） |
| answer | 25 | 两场加起来 15 条追问，各一次作答，跨档合并去重 |
| verdict | 20 | 两场各 5 条主张 × 三档追问强度（追问记录进提示词，档位不同提示词就不同） |
| suggest | 12 | 4 条需要改写的建议 × 三档改写风格（「站得住」的那条跳过；第二场五条全站得住，不产生建议） |
| counter | 4 | 一轮「轮到你了」的合成反问，跨档合并去重 |

跑完一场离线演示的权威数字（`.tmp/doc-stats2.mts` 直接调 `runAndPersist` 实测，常规档）：

| 指标 | 值 |
|---|---|
| 耗时 | 46 ms |
| 轮数 / 主张 / 追问 / 判定 / 建议 | 3 / 5 / 10 / 5 / 4 |

| 指标 | 值 |
| --- | --- |
| 被锚定机制拦下的主张 | 1 |
| 判定分布 | 站得住 1 · 表述过度 1 · 证据缺失 2 · 无法回答 1 |
| token | 输入 27,050 · 输出 4,122 · 思考 724（合计 31,172） |
| 反问 | 4 句（依据 JD 2 · 依据简历原话 2） |
| 每位面试官追问数 | tech 4 · hr 3 · hiring_manager 3 |
| 岗位要求对照 | 7 条要求：答得上 1 · 答得含糊 3 · 简历里没写 3 |
| 模型调用 | 33 次，回放 33 次（一个网络包都没发） |
| 事件 | 24 条（阶段事件可断线续传，分片不进事件表） |

### 4.4.3 卡带是合成的，而且必须被证明是合成的

这 149 盘卡带由 `createDemoScriptedClient` 用**确定性脚本**生成，不是真实模型的输出。这一点在文档里、在 `fixtures/README.md` 里都写明，并且有三重可核查的痕迹：

| 痕迹 | 怎么核查 |
| --- | --- |
| 每盘卡带的 `model` 字段都是 `scripted-demo` | 打开任意一个 JSON 就能看到 |
| `recordedAt` 是固定常量 `2026-10-26T00:00:00.000Z` | 因此重跑 `npm run seed:demo` **逐字节相同**，git status 干净 |
| usage 是按字符数折半的估算值，不是计费数据 | 生成器里有明确注释 |

为什么必须这么做？因为**卡带是要提交进公开仓库的。**真实模型输出 + 真实简历 = 隐私泄漏。 因此 `fixtures/README.md` 里写了一条红线：只允许用示例简历录制卡带。

### 4.4.4 提示词漂移门禁

卡带最大的风险是「代码改了提示词，卡带没重录，演示静默变味」。守住它的是一条测试：

```
packages/server/test/demo-cassette.test.ts
  用仓库里已提交的 fixtures/llm 起一个真实 app，
  POST /api/sessions（useDemo）→ POST /run?stream=0
  断言：200 / 主张 5 / 被拦 1 / 判定含 unanswerable 与 solid / 建议 ≥ 1 / 反问 4 / 双锚点非空
```

这条测试跑的是**已提交的素材**，因此任何人改了提示词而忘了 `npm run seed:demo`，提交门禁立刻变红。 它同时也是一份「离线演示确实能跑完」的回归证据。

### 4.4.5 想联网跑真模型怎么办

```
cp .env.example .env       # 填 LLM_BASE_URL / LLM_API_KEY / LLM_MODEL
# .env 里设 CROSSEXAM_MODE=live
npm run start
```

服务启动横幅会明确打印当前模式与「是否已载入 .env」。实测在 DeepSeek 上跑通（见 5.5 节）。

## 4.5 演进链路：250 余次提交是怎么来的

### 4.5.1 规则写在仓库里

`AGENTS.md` 第 2 节把赛题的提交要求变成了可执行的工程规则：

| 规则 | 内容 |
| --- | --- |
| 粒度 | 一个提交 = 一个可独立说明的改动。禁止 `git add -A && git commit` 式大杂烩 |
| 判据 | 提交前先问：**这个提交对应哪一次 AI 对话？** 答不上来就说明粒度不对 |
| 格式 | Conventional Commits + 中文简述（不超过 50 字）+ 正文写「为什么这么改」 |
| 署名 | **禁止** `Co-Authored-By` 等 AI 署名尾注（GitHub 会把它们解析成账号并计入贡献者列表） |

### 4.5.2 阶段与提交的对应

| 阶段 | 内容 | 提交数（约） |
| --- | --- | --- |
| P0 | 仓库初始化、工程约定、DSH 会话导出器 | 5 |

| 阶段 | 内容 | 提交数（约） |
|---|---|---|
| P1 | 领域内核：锚定 / 切分 / 预算 / 契约 / schema；LLM 适配层 | 17 |
| P2 | 五阶段编排、存储、HTTP/SSE、报告、示例简历、冒烟 | 15 |
| P3 | 前端：包骨架、HTTP 契约、事件折叠、三栏界面、单端口托管 | 7 |
| P4 | 截图器、意图契约可见、复现预演、README 复现指南、流式吐字 | 30+ |
| P5 | 技术文档、Prompt 链刷新、发布清单 | 8 |
| P6–P7 | 判定口径 / 思考可视化 / 报告三形态 / 12 项可用性优化（导入、工作室、采纳改写、零门槛启动） | 60+ |
| P8 | 岗位要求审阅：1–8 份 JD、双锚点、schema v7 表重建 | 15 |
| P9 | 行业模板库：12 个行业 × 3 个岗位，一键采用幂等 | 10 |
| P10 | 追问深化：按类别看短板 / 追问强度三档 / 补充说明 / 轮到你了 | 30+ |

### 4.5.3 一个具体例子：一次 Bug 修复对应哪一次对话

以「修改建议全部失败」这个 bug 为例，它对应的提交链是：

```
c8a6e90 fix(shared): 修改建议不再要求模型自报主张编号
```

提交正文写的是「为什么」：提示词从未要求模型输出 claimId，而 schema 把它列为必填； 合成卡带恰好写了这个字段，于是离线全绿、真实模型全红。这类信息只有写进提交正文才留得下来。

## 4.6 Prompt 链：过程证据不是事后补写的

赛题要求提交「≥3 个核心功能迭代的完整 Prompt 链（提问、AI 建议、如何引导修 Bug）」。

本项目的做法是**从第一行代码起自动导出**：

| 环节 | 实现 |
|---|---|
| 原始数据 | DSH 把每一次会话写成 zstd 多帧压缩的 JSONL，落在 `%APPDATA%\dsh-desktop\harness\sessions\<工作区>\<会话>\session.v4.jsonl.zstd` |
| 解析 | `tools/dsh-export` 逐帧解压（`zstdDecompressSync` 只解第一帧，必须扫描魔数 `28 B5 2F FD`），按 turn 组织成时间线 |
| 脱敏 | 自动识别 6 类密钥格式（OpenAI / Anthropic / GitHub / AWS / Bearer / KEY=value 赋值），命中即替换 |
| 过滤 | 只把 `source.kind === 'user'` 的消息当真人提问，系统注入（runtime-context / skill-catalog / plugin:hindsight / time-context / agent-instructions）全部丢弃 |
| 分页 | 超过 40 万字符自动按**条目**边界切分（不是按轮次，因为单轮可能 144 万字符），每份都能在网页上直接渲染 |
| 产物 | `docs/prompt-chain/` 下 30 份分片，共约 8.0 MB，覆盖整个开发过程 |

这条链带来三个别的方式给不了的性质：

| 性质 | 含义 |
|---|---|
| 不可事后美化 | 快照是会话日志的直接产物，改一个字就会和日志对不上 |
| 包含失败 | 走错的路、被推翻的方案、修 bug 的过程都在里面，不是只挑成功的展示 |
| 可复跑 | `npm run export:prompt` 一条命令重新生成，不需要手工整理 |

## 4.7 本章要点

- 复现路径有两条：双击 `启动.cmd`（零门槛）或四条命令（120.8 秒，实测，由 `npm run repro` 生成，5 倍余量）。
- 零原生依赖是硬约束，至今没有破例。
- 离线回放让「没有 Key 也能看到完整效果」成立，149 盘合成卡带 + 一条漂移门禁守着它。
- 提交粒度规则写在仓库里，250 余次提交每一次都能对应到一次具体的开发动作。
- Prompt 链是会话日志的直接产物，自动导出、自动脱敏、自动分页——过程证据不是作文。

# 第五章　实测证据

这一章只放**可复跑的数字**。每条结论后面都标了它是怎么测出来的；凡是无法复跑的观察（例如「真实模型偏保守」），都会明确写成观察而不是结论。

## 5.1 门禁现状

```
npm run verify
  1/4 typecheck   tsc --noEmit（根 + shared + server + web）
  2/4 test        63 个测试文件 / 659 条断言
  3/4 smoke       9 项真进程冒烟
  4/4 build       vite 8.3.5 / 44 modules / CSS 29.07 kB / JS 340.75 kB（gzip 104.70 kB）
  exit 0
```

测试分布：

| 包 | 测试文件 | 覆盖重点 |
|---|---|---|
| shared | 15 | 锚定的四种拒绝、归一化映射、预算五道闸门、契约渲染确定性、schema 解析、自建面试官草稿（18 条）、采纳改写与标记坐标（11 条）、岗位模板库 36 条的合规与指纹（8 条）、岗位要求的锚定与归并（14 条）、短板的分组排序与结论（11 条）、追问强度三档只换预算（10 条）、补充说明的长度与插入点（10 条）、反问依据的锚定（12 条）、并排对照的上下文行（5 条）、改写风格三档的文案与取值（6 条）、两场主张的相似度配对与判定位移（17 条） |
| server | 24 | 九阶段编排、闸门顺序、SSE 收尾、迁移（含 v6→v7 表重建与 v8 / v9 加表）、卡带、流式、阶段路由、报告、简历导入（PDF / docx / 文本共 11 条）、运行模式与模型清单、面试官库与契约优化（9 条）、采纳改写与三种导出（14 条）、补充说明的落库与采纳插入（14 条）、岗位模板库的只读与幂等采用（6 条）、多 JD 的抽取 / 归并 / 对照与双锚点落库（6 条）、三档预算与档位落库（7 条）、反问的锚定与两份报告（5 条）、报告里的改前改后对照（3 条）、改写风格的落库 / 下发 / 换档不动指纹（7 条）、版本对比接口与两份报告的「比上次」（11 条） |
| web | 17 | 文案漂移与判定口径、高亮切分（含「聚焦压住已改写段」）、事件折叠（直播 vs 历史）、流式分片解析、报告网址、配色主题、新手教程文案、工作室常量与 shared 对账（8 条）、采纳栏状态（10 条）、模型提示文案的四种态（6 条）、模板分组与采用提示（8 条）、JD 列表编辑（7 条）、要求对照的派生值与分组（9 条）、短板派生值（5 条）、反问事件折叠（4 条）、「比上次」的四种状态与逐条位移文案（6 条） |
| tools | 7 | 分页、zstd 多帧、脱敏、Markdown 渲染、零门槛启动器（版本判定 / 该走哪几步 / 快捷方式脚本，共 12 条） |

测试文件总数 15 + 24 + 17 + 7 = 63；`npm run e2e:ui` 另有 **132 项**真浏览器行为断言（不进 verify，因为它要一台装了 Edge/Chrome 的机器）。

## 5.2 三条最有说服力的测试

### 5.2.1 直播与历史必须给出同一个界面

`packages/web/test/state.test.ts` 把同一串 15 个事件喂给两条路径：

```
路径 A：sandboxReducer + {type:'event'}        （模拟 SSE 直播）
路径 B：stateFromHistory(events)               （模拟刷新后从库里读）
断言：claims / probes / budget / errors / interviewers / rejectedClaims / eventCount 完全一致
      只有 phase 不同（'running' vs 'done'）
```

这条测试守的是产品承诺「刷新后看到的和刚才实时看到的是同一场」。它一共有 17 条断言，其中还覆盖了：预算事件让主张关停、作答报错在卡片上显示、空追问轮自己补提示、未知 claimId 与重复 probeId 被丢弃、致命错误把 phase 置为 failed。

### 5.2.2 提示词必须是确定性的

`packages/server/test/orchestrator-extract.test.ts` 有一条：

```
同一个输入渲染两次提示词 → 逐字节相同
```

理由不是洁癖：卡带按**提示词指纹**寻址，提示词抖一字节，整盘离线回放就失效。因此所有阶段的提示词全部是纯函数，不含时间戳、不含随机数、不依赖 `Object.keys` 的顺序。

### 5.2.3 锚定不采信模型给的坐标

同文件另一条：给模型一个**错误的 start/end**，但引文是对的， 断言最终锚点是程序用 `indexOf` 重新定位出来的区间。 这条把「证据锚定」从一句口号变成了可验证的行为。

## 5.3 全接口验收：78 项（一次性脚本，端点此后又长了一倍）

`.tmp/api-matrix.mjs`（一次性验收脚本，自起服务、自建临时库）覆盖：

> 诚实的边界：这份脚本**没有进仓库**（它在 `.tmp/` 下），测的也是当时的端点集。此后又加了简历导入、 面试官库、岗位模板库、采纳改写与多 JD 这几组接口，而它们现在的覆盖在 `npm run test`（server 24 个文件） 与 `npm run e2e:ui`（132 项真浏览器断言）里。把全接口矩阵做成**可复跑的命令**列在 P13，不在这里假装它还新鲜。

| 类别 | 项数 | 例子 |
|---|---|---|
| 健康 / 示例 / 契约 | 12 | 契约指纹格式、`?panel=tech,cto` 返回 400 并列出可选值 |
| 建档错误分支 | 4 | 简历太短（少于 30 字）返回 400 且中文提示 |
| 完整跑一场 | 18 | 轮数 3、主张 5、追问 10、判定 5、建议 4、被拦 1、usage 输出 2,267 |
| 落库校验 | 9 | 事件 18 条且 seq 连续、`?from=10` 续传 8 条、账本 calls=29 |
| 报告 | 6 | 四个章节齐全、Markdown 可直接下载 |
| SSE | 5 | 含 `stream` 分片且**分片不进事件表** |
| 静态托管 | 4 | 首页 200、SPA 深路径回退、未知接口中文 404 |

结果：**78 PASS / 0 FAIL。**

## 5.4 真浏览器实测：控制台 0 报错

不装 Playwright，用系统 Edge + CDP + Node 自带 WebSocket 驱动（`tools/ui-shots/shots.mjs`）：

| 项 | 结果 |
|---|---|
| 截图 | 24 张，全部由命令生成（含新手教程、浅色配色、面试官工作室、采纳改写、岗位模板库、岗位要求对照、按类别看短板、追问强度、补充说明、反问清单与改前改后对照） |
| 控制台 | **0 条报错 / 0 条警告**（收 Runtime.exceptionThrown、consoleAPICalled 的 error 与 warning、Log.entryAdded） |
| 流程 | 点「慢放」→「一键示例」→「开始质询」，跑到判定与建议全部出现 |
| 界面数字 | 调用 33 次 · 回放 33 · token 31,172 · 事件 24 · 追问 10 |
| 与后端一致性 | 界面判定条数与服务端 `verdicts` 条数一致 |

截图清单（`docs/screenshots/`）：

| 图 | 内容 |
|---|---|
| 01-intake.png | 空状态：四条机制逐条说明 + 意图契约三条 pending |
| 02-demo-loaded.png | 载入示例简历 |
| 03-running.png | 直播中：三位面试官同时追问 |
| 04-live-output.png | **实时输出栏：模型正在写的人话**（原始输出是 JSON，界面已还原） |
| 05-thinking.png | **同一栏的「思考 + 输出」两行**：斜体是思考分片，正下方是正文分片 |
| 06-done-top.png | 跑完总览 |
| 07-verdicts.png | 判定卡片，含「这条判定不是模型直接给的」 |
| 08-resume-focus.png | 点引文后右栏聚焦到「第 81–108 字」 |
| 09-history.png | 刷新后从历史恢复 |
| 10-contracts.png | 意图契约摊开 + 徽章「与运行时一致 · 指纹 b2171c82」 |
| 11-system-prompt.png | 该面试官实际拿到的 System Prompt 原文 |

| 图 | 内容 |
| --- | --- |
| 12-help.png | 第一次打开自动弹出的新手教程：四步上手、三种模式对照表、隐私口径 |
| 13-light.png | **浅色配色**：白天/投影仪/打印稿上看得清；判定卡片里带「为什么这样判」与「判据展开」 |
| 14-report.png | **服务端渲染的可打印报告**：元信息表、过程说明、带判定配色的主张与判定——PDF 就是它经浏览器打印得到的 |
| 17-adopt.png | **采纳改写后的右栏**：「改完的简历 470 字 · 改动 4 处（绿色）」+ 已改/待定/改不动/本次勾选四枚计数 + 三个导出入口（.txt / PDF / Word） |
| 18-templates.png | **岗位模板库**：12 个行业 × 3 个岗位，点一下把岗位名、JD 与示例简历（258 字）一起填好，并把模板自带的面试官装进阵容 |
| 19-requirements.png | **岗位要求对照**：中栏按「答得上 / 答得含糊 / 简历里没写」分三组，每条要求一张卡（要求正文、引文、「为什么这么判」、对应哪条主张）；右栏在简历原文与岗位要求原文之间切换，点引文聚焦到 JD 的第 28–48 字 |
| 20-shortfall.png | **按类别看短板**：同一份判定换个角度摆——结论句 + 四行类别（各欠几条、欠在哪个判定上）+「最该先补的 3 处」，点一下跳回简历原文 |
| 21-difficulty.png | **追问强度三档**：挑刺档选中时，契约卡片上的预算与指纹同时换掉（4 轮 / 每人 2 次 / 全场 80 次 · 指纹 cf9c6457） |
| 22-supplement.png | **补一句真话**：用户手写的一句事实挂在主张卡片上（不写进简历原文），点「确认采纳」时它才作为新的一行进简历 |
| 23-counters.png | **轮到你了**：4 句可以反问 HR 的话，每句带「依据岗位要求 / 依据简历原话」徽章与可点的引文，回答下注明「沙盘模拟，不是真 HR 在答」 |
| 24-diff.png | **改前改后对照**：采纳之后点「并排看改动」，每条改动一行——左边是简历原文里的那一段，右边是改完后的那一段；改动之间没动过的原文也作为上下文行摆进来，所以看得出改动落在什么位置 |

## 5.5 真实模型实测（DeepSeek，live 模式）

这一节的意义是：**证明离线演示不是靠卡带撑起来的空壳**，接上真模型确实能跑。

### 5.5.1 端到端结果

| 项 | 值 |
| --- | --- |
| 模型 | deepseek-chat（OpenAI 兼容协议） |
| 一场耗时 | 20.0 s |
| 主张 / 追问 / 轮数 / 建议 | 10 / 18 / 3 / 9 |
| token | 输入 47,941 · 输出 8,355 · 缓存命中 7,424 |
| 判定 | 10 条「证据缺失」（判定口径调整后的实测，见 5.6 第 5 条） |
| 被锚定机制拦下的主张 | 0（真实模型的引文质量比预期好） |

### 5.5.2 真实追问的质量抽样

下面这条是真实模型对「首屏加载从 2.4s 降到 0.8s」的追问，未经任何修改：

> 2.4s 和 0.8s 这两个首屏加载数字是怎么测出来的——用什么工具、在什么网络条件和设备上、取的是几次采样的什么统计口径?

求职者 Agent 的回答（同样未经修改）：

> 这个……简历里只写了「首屏加载从 2.4s 降到 0.8s」这个结果，具体用什么工具测的、什么网络条件和设备、采样几次取什么统计口径，简历里没有写，我这边没法给你具体细节。

**这一问一答就是整个产品的价值主张**：它不是「AI 帮你写简历」，而是「AI 找出你答不上来的那句」。

### 5.5.3 真实模型的流式分片

live 模式下的一场实测：**5,814 个分片 / 29 路 channel / 0 片进入事件表 / 21 条阶段事件。** 接上思考模型（`CROSSEXAM_REASONING_MODEL=deepseek-reasoner`，只路由 verdict）之后再测一场：**2,355 片思考分片 / 8 路 `verdict:*` 通道 / 账本 `reasoningTokens` > 0 / 整场 22.7 s**—— 思考与正文分走两个 `kind`，但同样不进事件表。 这组数字同时证明了三件事：流式真的在流、分片确实被隔离在事件日志之外、阶段事件的规模稳定在二十条左右。

### 5.5.4 两条诚实的观察（不是结论）

| 观察 | 说明 |
|---|---|
| 真实模型抽的主张比合成脚本多 | 361 字简历抽 8–10 条，合成脚本固定 5 条；真实模型还会把「2023.06 - 2025.03」这类时间线也抽成主张 |
| 判定口径调完之后，「无法回答」几乎不再出现 | 求职者 Agent 只依据简历作答，它天然说不出「我不会」，所以「无法回答」本就该是少数；数字类主张更容易落在「证据缺失」（简历没写口径，改简历即可）。代价是区分度下降：这一场 10 条判定全是「证据缺失」 |

### 5.5.5 全链路真机验收：真浏览器 + 真联网 + 用户的真实 PDF 简历

上面几节测的是单个环节。最后做了一次**端到端**验收：`.tmp/live-full-all.mjs` 真起服务、真调 DeepSeek、把 `C:/Users/xinyi/Desktop/第三版简历.pdf` 拖进导入框（解析出 1006 字），再用真浏览器把从投料到导出报告的每一步都点一遍。

| 轨道 | 形态 | 结果 |
|---|---|---|
| 回放 | `npm run e2e:ui`（无头 Edge） | **132/132**，控制台 0 报错 |
| 联网 + 真实简历 | 无头 Edge + 真 API + PDF 导入 | **32/32**，整场 19–28 s |
| 出图 | `npm run shots` | 24 张，控制台 0 报错 |

联网那一场：12 条主张全部锚定 / 12 条判定 / 18 次追问 / 3 轮 / 8 条建议，「改 8 处 1209 字」，其中 1 处改不动并写出了原因；报告 Markdown 16,326 字符、可打印 HTML 6,191 字符、Word 6,108 字符。打开思考模型后再跑一场，思考分栏最长一次读到 **221 字**——注意这个数字是 500 ms 轮询采样取的最大值，只在某一瞬间读一次必然为 0，因为思考分栏只在 verdict 阶段出现。

这次验收抓到的四个缺陷全在**脚本**里（隐藏的 `type=file` 抢到了 `input[0]`、残留 Edge 占着调试端口、`waitText` 抓到 verdict 中间态导致库里还是 0 行、`view.verdicts` 的字段形状猜错），而唯一的产品问题也正是「等待写法不可靠」的同源问题——过程态与结论态共用一个元素（F48、D41）。**测试脚本的可靠性和产品的可靠性是同一种能力；脚本假 PASS 和产品真 BUG 一样要当缺陷处理。**

### 5.5.6 P10 四项追问深化：每一项都有自己的界面断言

P10 之后 e2e 从 88 项涨到 **117 项**；P11 的 B2 加了 4 项、B1 加了 5 项，现为 **125 项**。新增的全是「界面读出来的结论必须与服务端算出来的一致」，不是「按钮存在」：

| 项 | 界面断言（节选） | 实测 |
|---|---|---|
| 按类别看短板 | 组数 = 判定过的类别数；名单条数 = min(短板数, 3) | 4 组 / 4 个类别；名单 3 条 / 服务端 4 条短板 |
| 追问强度三档 | 换档后预算与契约指纹都变，切回来必须回到原来那个指纹 | 042b385f → cf9c6457 → 042b385f |
| 补充说明 | 存下之后编辑区收起，点「改一句」才重新出现；删掉后卡片回到干净态 | 存 → 收起 → 改 → 删 |
| 轮到你了 | 引文数 = 服务端反问数；两种依据都标出来源；点引文那一侧原文聚焦到依据那段 | 4 句 / 2 句依据 JD + 2 句依据简历 / 第 81–108 字 |

三档卡带另有一条服务端测试守着：温和 / 常规 / 挑刺三档在**零网络**下都能跑完，且预算真的流进了契约（每主张 2 / 3 / 4 轮，每人每主张 1 / 2 / 2 次）。

## 5.6 五个在实测中抓到的真 Bug

下面五个 bug 都不是设计阶段发现的，是**真跑起来才暴露**的。它们比任何架构图都更能说明这套工程是真的在运转。

| 序号 | 现象 | 根因 | 为什么之前没抓到 |
|---|---|---|---|
| 1 | 真实模型下 9 条修改建议全部失败 | schema 要求 `claimId` 必填，提示词从未要求输出它 | 合成卡带恰好写了这个字段，把矛盾完整掩盖 |
| 2 | 同进程第二场会话 500 | claim 表全局主键，而主张编号每场从 C1 重新开始 | 每个测试都只跑一场 |
| 3 | 真起服务第一次写入报 `no such table: session` | 建表靠调用方显式调 migrate，测试都调了、真进程没调 | 同进程 `app.request()` 测试不经过启动路径 |
| 4 | live 实测「0 分片」 | 端口上挂着**流式功能落地之前**的旧进程 | 没有验证「测的是谁的进程」 |
| 5 | 判定徽章与模型写的理由互相矛盾 | 代码级收敛把「全体未答」一律改判「无法回答」，吃掉了模型判的「证据缺失」 | 合成卡带的作答脚本恰好都答得上，离线演示永远不触发这条规则 |

每一个都变成了规则或门禁：

| Bug | 变成的规则 |
|---|---|
| 1 | 给模型输出造夹具时，唯一真相是**提示词里的输出要求**，不是 zod schema（F23） |
| 2 | 复合主键 + 迁移冻结在自己时点（F24） |
| 3 | `openDatabase` 连接即建表；门禁加入真进程冒烟（F11） |
| 4 | 一次性实测起服务前必须自检端口为空、核对写入的是自己的库（F22） |
| 5 | 代码级收敛只做「防过于乐观」，责任归属交回模型；两档判据写成可判别的「责任在谁」（F31） |

### 5.7 本章要点

- 门禁四道：类型、逻辑、**产品能跑**、能构建。第三道不可替代。
- 界面点测 125 项全绿；真浏览器 24 张截图、控制台 0 报错；另有 78 项一次性全接口验收（脚本未进仓库，见 5.3）；

报告三种形态另有真机验收（`.tmp/report-check.mjs`，12/12）：HTML 经 Edge 打印得到 438305 B / 6 页 PDF，界面三个按钮分别指向 `.md` / `.html?print=1` / `.doc`。

- 真模型跑得通：20 s、10 条主张、18 次追问，追问质量可直接引用于演示视频。
- 五个真 bug 都变成了规则，而不是被悄悄修掉——这才是「演进链路」的实质。

---

# 第六章　局限、取舍与后续工作

一份技术文档如果只讲做成了什么，它的可信度反而更低。这一章写清楚：**什么没做、为什么没做、如果继续做会先做哪一件。**

### 6.1 已知局限

| # | 局限 | 影响 | 现状 |
|---|---|---|---|
| 1 | 求职者 Agent 只依据简历作答，不使用用户的真实记忆 | 它答不上来的问题，用户本人**可能**答得上来 | 刻意如此：测的是「简历这句话能不能自证」，不是「你本人会不会」；报告里也写明 |
| 2 | 判定区分度不足：真实模型下几乎全判「证据缺失」 | 看不到「表述过度 / 与原文矛盾 / 无法回答」这些信息量更大的档位 | 口径已修正为「责任在谁」（见 2.6.1）；但求职者 Agent 只依据简历作答、天然说不出「我不会」，所以「无法回答」稀少是结构性的 |
| 3 | 追问策略依赖单一模型的角色扮演 | 三位面试官的提问风格差异不如真人明显 | 契约层已把差异结构化（tactics / forbidden 各不相同），但同一模型扮演多个角色仍有同质化倾向 |
| 4 | 只支持中文简历 | 英文简历的锚定与切分未验证 | 锚定逻辑本身与语言无关，但 segment 的小节识别按中文标题写死 |
| 5 | 当初判断「解析 PDF 会引入原生依赖」，与「10 分钟复现」冲突 | 一度只能粘贴纯文本 | ✅ 已推翻：纯 JS 的 `unpdf` 出现后这条不再成立，现支持 PDF / Word / 文本导入，字节不落盘（ADR-0007） |
| 6 | 单机单用户，无鉴权 | 不能直接部署到公网 | 产品定位是本地优先工具，简历属隐私数据，**不上传任何第三方存储** |

### 6.2 三个明确的取舍

#### 6.2.1 抄一份文案，而不是拆包

web 不 import shared（shared 经 util.ts 依赖 `node:crypto`），代价是判定标签与人设文案要抄一份。正确做法是把 shared 拆成零依赖 core + 带 IO 的 adapter，但那是一次全仓库重构；本项目用**漂移测试**（`toBe` 断言同一个对象）替代，把风险压到「测试红了才发现」，而不是「重构期间功能停摆」。

#### 6.2.2 合成卡带，而不是录真实模型输出

真实模型输出 + 真实简历 = 隐私泄漏，而卡带要提交进公开仓库。因此演示卡带由确定性脚本生成， 并用三重痕迹标明是合成的（model 字段、固定 recordedAt、估算 usage）。 代价是离线追问质量不如真实模型（合成脚本 5 条主张，真实模型抽 8–10 条）；补偿是把真实实测的数字与追问原文写进 5.5 节，两者都能被核查。

### 6.2.3 分片不落库

流式分片走 SSE 但不进事件表，代价是刷新后看不到「刚才正在写的字」；换来回放不被几万条分片拖慢、日志不吵， 而且「能断线续传的只有阶段事件」这条边界很清楚（ADR-0004）。

## 6.3 如果继续做，先做这三件

| 优先级 | 事项 | 为什么是它 |
| --- | --- | --- |
| 1 | 让候选人 Agent 能说「我不会」，而不只是「简历里没写」 | 根治 6.1 第 2 条的判定区分度问题：现在它只能引用简历，所以「无法回答」结构性稀少 |
| 2 | 支持英文简历 | 锚定逻辑已与语言无关，只需放宽 segment 的小节识别 |
| 3 | 把 shared 拆成 core + adapter | 消除抄文案的技术债 |

原表里的另外两件——追问阶段可补一句事实、报告按主张类别统计——已在 P10 落地（见 3.6.10）。

这张表是「技术上最先该补的」；「产品上还能加什么」见 候选功能清单（六组二十余项，每项写明解决谁的问题、大概几小时、值不值得做）。

## 6.4 结语

这个项目想做的一件事，用一句话说就是：

> **在简历投出去之前，先让三位不同的面试官把它拆一遍，并且告诉你哪一句你答不上来。**

技术上它试了三件不太常见的事：

1. **把「提问必须有证据」做成程序闸门**（不是提示词里的请求）——幻觉式提问在物理上进不来；
2. **把「追问次数」做成不可变账本**（不是模型自己数数）——死循环在设计上不成立；
3. **把「过程证据」做成会话日志的自动产物**（不是事后整理的文档）——250 余次提交、34 份 Prompt 链快照与本文档里每个数字都可追溯。

第三件事也是对赛题「过程可复现性」那条 25% 权重最直接的回答：过程不是事后描述的，是仓库里一条命令就能重新导出的。

---

**附：本文档自身的可复现性**

本文档由 `npm run docs:pdf` 从 `docs/tech/*.md` 生成：零依赖 Markdown 渲染（`tools/docs/md2html.mjs`）→ 无头 Edge 打印（`tools/docs/build-doc.mjs`）→ PDF；构建时统计页数，超过 30 页直接失败（赛题上限）。